



Information Systems for Big data

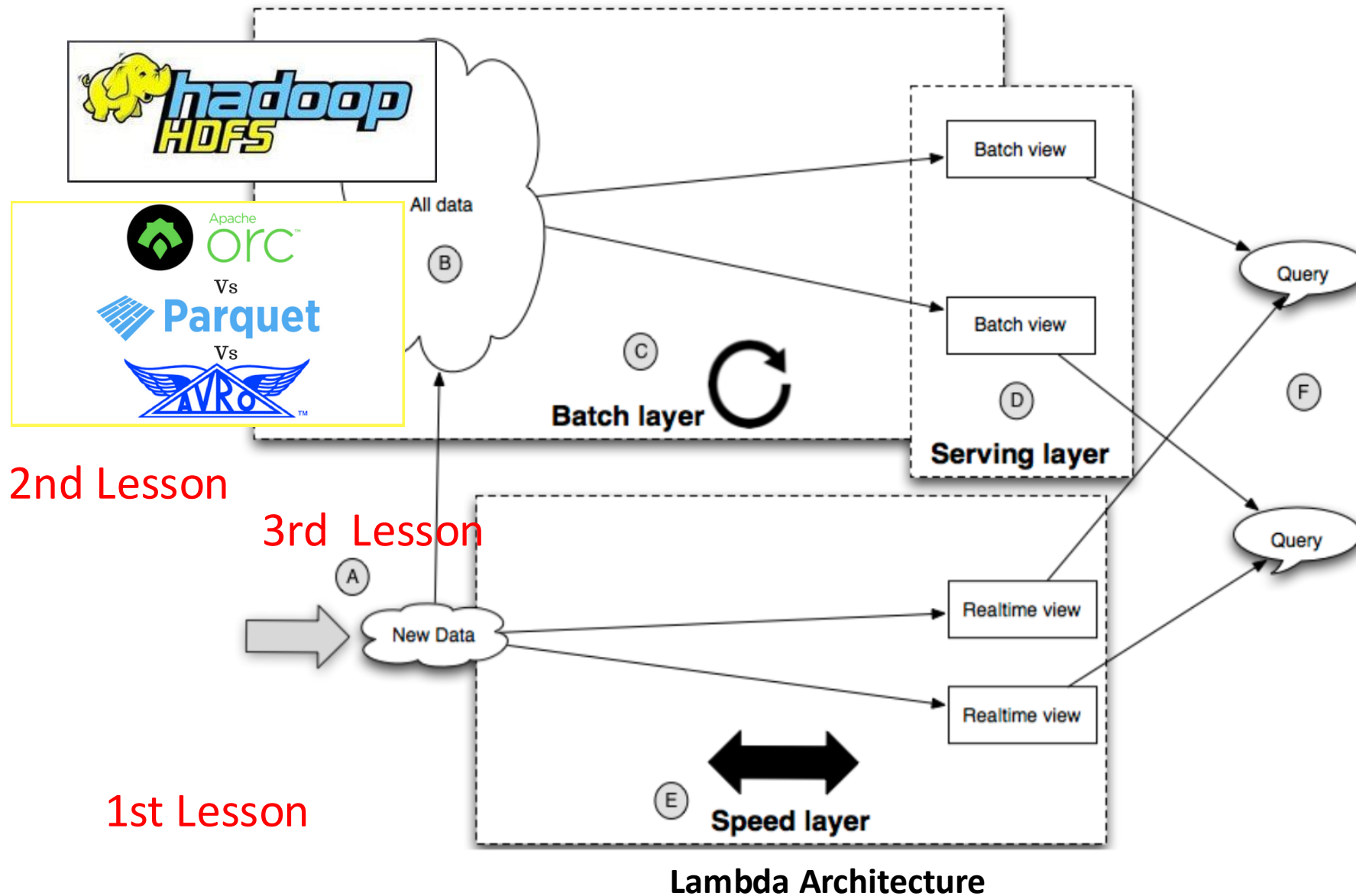
Prof. G. Fenza gfenza@unisa.it



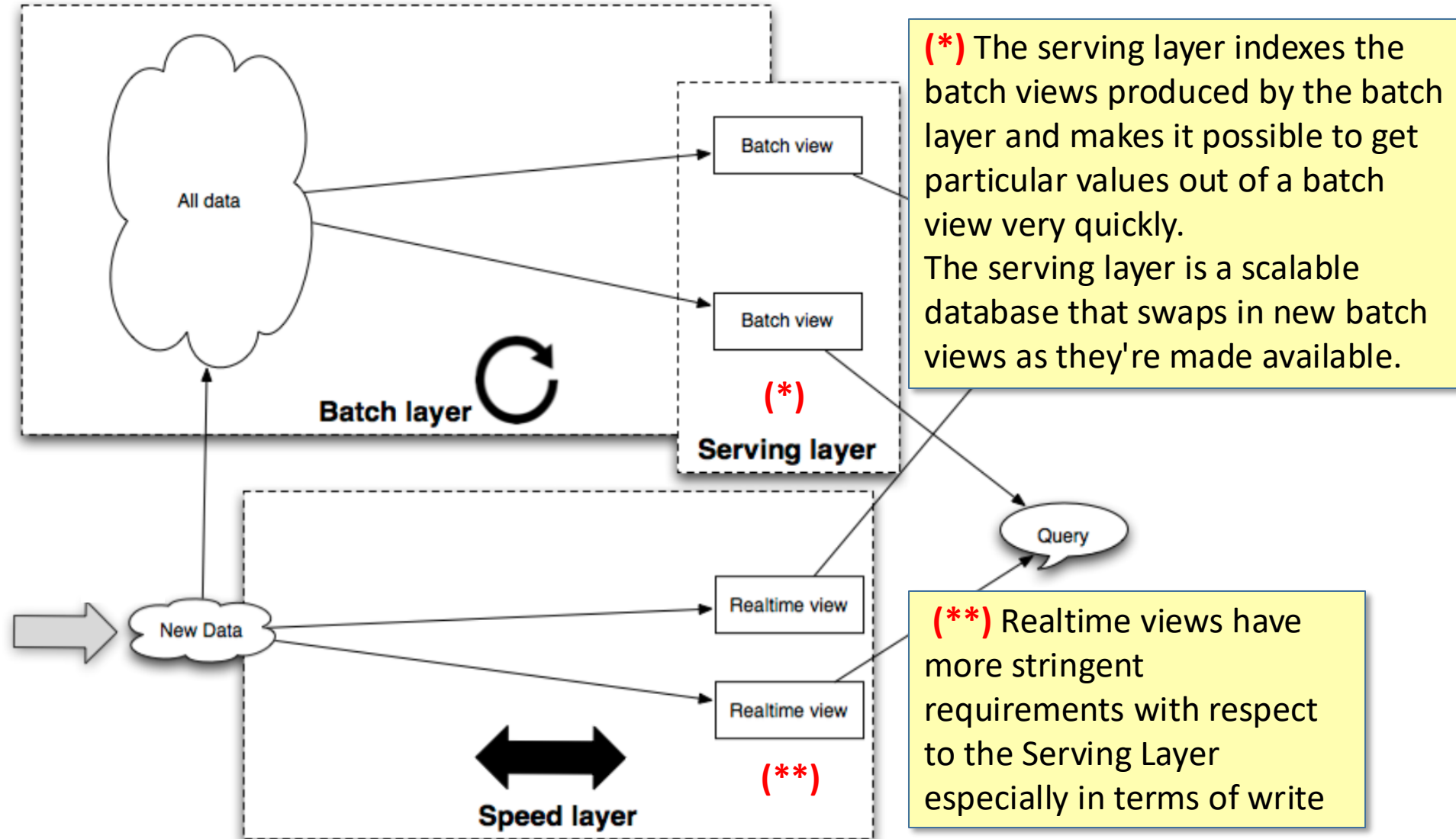
4th Lesson - NoSQL

- NoSQL and LAMBDA Architecture

Previous Lessons



The focus of the 3rd Lesson



Lambda Architecture

Consistency vs Efficiency

- 1 Data is normalized in the master dataset for compactness and consistency...

Batch layer



Master dataset

- 3 The batch views are continually rebuilt from the master dataset, so all changes are consistent across the batch views.

- 2 ... but is redundantly stored (denormalized) in the batch views for efficient querying.

Serving layer



Batch view



Batch view

Query:

"How many 2"

Normalized vs Denormalized

Batch Layer – Master Data Storage Requirements



- **Write** — The only write operation is to add new species of data, in addition let's consider that the batch layer stores the complete dataset (terabytes, petabytes, and more)
 - Efficient appends of new data.
 - Scalable storage
- **Read** — Support for parallel processing.
- **Both** — You may choose to compress your data to help minimize your expenses. Let's consider that it's critical to guarantee the immutability property.
 - Tunable storage and processing costs.
 - Enforceable immutability

Serving Layer – Storage Requirements



- **Batch writable** — The batch views for a serving layer are produced from scratch. When a new version of a view becomes available, it must be possible to completely swap out the older version with the updated view.
- **Random reads**—A serving layer database must support random reads, with indexes providing direct access to small portions of the view. This requirement is necessary to have low latency on queries.
- **Scalable** — A serving layer database must be capable of handling views of arbitrary size. As with the distributed filesystems and batch computation, this requires it to be distributed across multiple machines.
- **Fault-tolerant**—Because a serving layer database is distributed, it must be tolerant of machine failures.

Speed Layer - Storage Requirements



- **Random writes**—To support incremental algorithms, it must also be possible to modify a realtime view with low latency.
- **Random reads**—A realtime view should support fast random reads to answer queries quickly. This means the data it contains must be indexed.
- **Scalability**—As with the serving layer views, the realtime views should scale with the amount of data they store and the read/write rates required by the application. Typically this implies that realtime views can be distributed across many machines.
- **Fault tolerance**—If a disk or a machine crashes, a realtime view should continue to function normally. Fault tolerance is accomplished by replicating data across machines so there are backups should a single machine fail.



File System

- Si è cominciato ad archiviare ed elaborare dati con i computer prima dell'avvento del database.
- Per essere utili per le applicazioni aziendali, i computer dovevano archiviare, manipolare e recuperare file di dati di grandi dimensioni.
 - A tale scopo sono stati sviluppati sistemi di elaborazione di file informatici: **file system**.
- Es: File System
 - I dati possono essere memorizzati in file excel, il file excel vengono gestiti come tutti gli altri file

Intro caso Pine Valley Furniture Company



- Pine Valley Furniture Company (PVFC) produce mobili di alta qualità e li distribuisce ai negozi al dettaglio in tutta la nazione.
- Il cliente invia gli ordini a Pine Valley Furniture con uno dei vari mezzi come telefono, posta, fax o moduli elettronici via Internet.
- Questa società ha inizialmente utilizzato sistemi basati su computer per:
 - l'elaborazione degli ordini
 - l'emissione delle fatture
 - il libro paga per il personale

- In tutte le applicazioni hanno utilizzato il tradizionale approccio di elaborazione dei file.
- Si sono concentrati sulle esigenze di elaborazione dei dati dei singoli reparti (dipartimenti).
- Ogni programma applicativo o sistema sviluppato è progettato per soddisfare le esigenze del particolare reparto o gruppo di utenti richiedente.
- Quindi non esiste una mappa, un piano o un modello generale per guidare la crescita delle applicazioni.

PVFC – file system



- Le applicazioni informatiche (basate sull'approccio di elaborazione dei file) più importanti per PVFC sono:
 - Evasione degli ordini,
 - Fatturazione (Invoicing)
 - Libro paga (payroll)

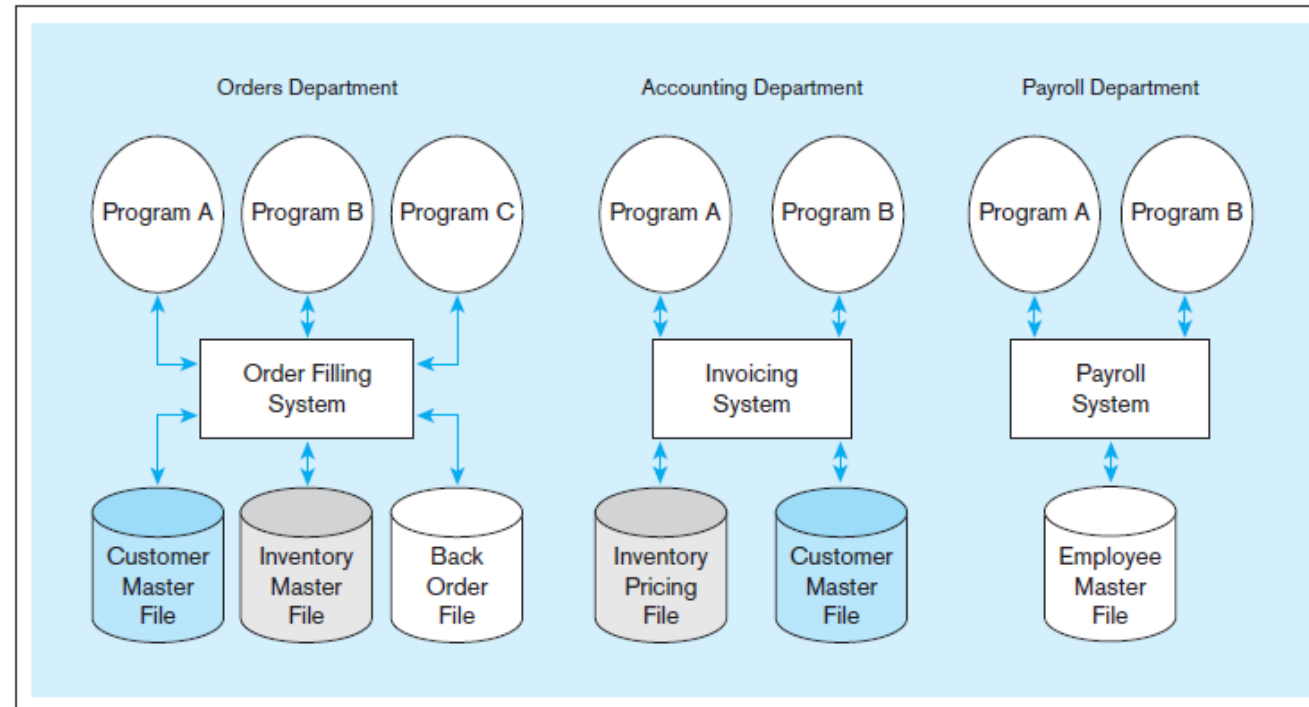


FIGURE 1-2 Old file processing systems at Pine Valley Furniture Company

La figura illustra gli svantaggi dell'utilizzo dell'approccio di elaborazione dei file in PVFC (v. slide succ.)

PVFC – file system



- Si noti che esiste una duplicazione di alcuni dei file utilizzati dalle tre applicazioni, tipico dei sistemi di elaborazione dei file.

Dati Duplicati

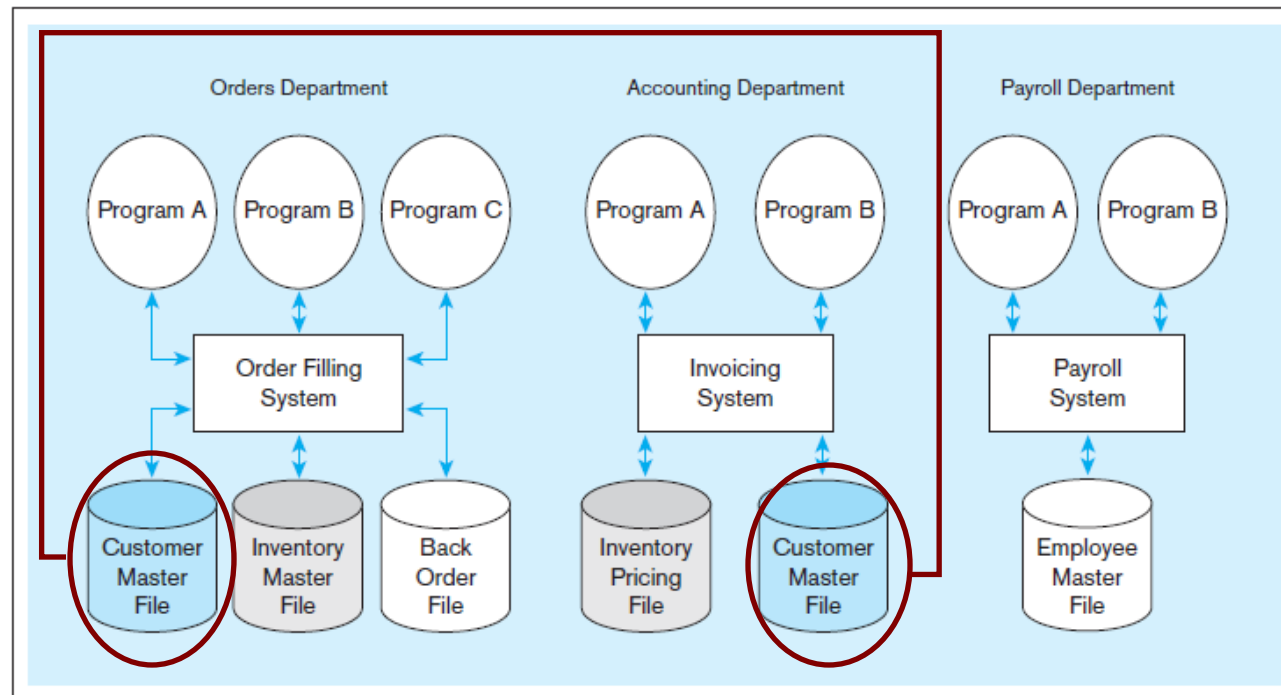


FIGURE 1-2 Old file processing systems at Pine Valley Furniture Company



Normalizzazione

Normalizzazione dei dati



- strumento per convalidare e migliorare una progettazione logica:
 - soddisfare determinati vincoli per evitare inutili duplicazioni di dati
- decomposizione delle relazioni con le anomalie per produrre relazioni più piccole e ben strutturate

Relazioni ben strutturate



- Relazione che contiene una ridondanza minima dei dati e consente agli utenti di inserire, eliminare e aggiornare righe senza causare incoerenze nei dati
- L'obiettivo è evitare anomalie

- è una tabella di dati bidimensionale con un determinato nome
- è costituita da righe (record) e colonne (attributo o campo)
- per essere qualificata come relazione:
 - Deve avere un nome univoco.
 - Ogni valore di attributo deve essere atomico (non multivalore, non composito).
 - Ogni riga deve essere univoca (non può avere due righe con esattamente gli stessi valori per tutti i relativi campi).
 - Gli attributi (colonne) della tabella devono avere nomi univoci.
 - L'ordine delle colonne deve essere irrilevante.
 - L'ordine delle righe deve essere irrilevante.

NOTA: tutte le relazioni sono nella prima forma normale.

Esempio di relazione



- La relazione EMPLOYEE1 contiene i seguenti attributi che descrivono i dipendenti: EmpID, Name, DeptName e Salary.
- Le cinque righe della tabella corrispondono a cinque dipendenti.
- Se aggiungiamo un'altra riga di dati alla figura o modifichiamo uno qualsiasi dei dati nelle righe esistenti, è sempre la stessa relazione.
- L'eliminazione di una riga non modifica la relazione.
- In effetti, potremmo eliminare tutte le righe e la relazione EMPLOYEE1 esisterebbe ancora.

Esempio di relazione



EMPLOYEE1

<u>EmpID</u>	Name	DeptName	Salary
100	Margaret Simpson	Marketing	48,000
140	Allen Beeton	Accounting	52,000
110	Chris Lucero	Info Systems	43,000
190	Lorenzo Davis	Finance	55,000
150	Susan Martin	Marketing	42,000

Una relazione ben strutturata contiene una ridondanza minima e consente agli utenti di inserire, modificare ed eliminare le righe in una tabella senza errori o incoerenze.

EMPLOYEE1(EmpID, Name, DeptName, Salary)

- EMPLOYEE1 è una relazione ben strutturata. Ogni riga della tabella contiene i dati che descrivono un dipendente e qualsiasi modifica ai dati di un dipendente (come una modifica dello stipendio) è limitata ad una riga della tabella.
- EMPLOYEE2 non è una relazione ben strutturata (Se si esaminano i dati di esempio nella tabella, si noterà una notevole ridondanza.)

EMPLOYEE2

<u>EmpID</u>	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015

Esempio



EMPLOYEE2

<u>EmpID</u>	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015

E' una relazione?

Sì: righe univoche e senza attributi multivalore

Qual è la chiave primaria?

Composita: EmpID, CourseTitle

Questa è una relazione. Ma non è una relazione ben strutturata.

Anomalie da evitare



Anomalia di **inserimento**

- aggiunta di nuove righe → creazione dati **duplicati**

Anomalia di **eliminazione**:

- eliminazione di righe → **perdita** di dati necessari per altre righe future

Anomalia di **modifica**:

- modifica dei dati in una riga → le modifiche ad **altre righe** a causa della duplicazione

Regola generale: una tabella non deve riguardare più di un tipo di entità.

Esempio Anomalie



EMPLOYEE2

<u>EmplID</u>	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015
100	Margaret Simpson	Marketing	48,000	JAVA	6/19/2016

Eliminazione

Se eliminassi il dipendente 140 non saprei più che nella mia azienda si tiene un corso di Tax Accounting

Modifica

Se cambiassi il salario del dipendente 110 dovrei aggiornarlo ovunque

Inserimento

Se volessi inserire un nuovo corso, oppure un nuovo dipendente, dovrei mettere a Null tutti i campi per i quali non sono disponibili informazioni. Inoltre, creerei inevitabilmente dei duplicati

Anomalie in questa tabella



- **Inserimento:** non è possibile immettere un nuovo dipendente senza che il dipendente assuma un corso (o almeno campi vuoti di informazioni sul corso)
- **Eliminazione:** se rimuoviamo il dipendente 140, perdiamo informazioni sull'esistenza di corso di Tax Acc
- **La modifica:** l'aumento di stipendio al dipendente 100 ci costringe ad aggiornare più (2) record

Perché esistono queste anomalie?

Poiché in questa relazione sono presenti due temi (tipi di entità). Ciò comporta la duplicazione dei dati e una dipendenza non necessaria tra le entità.

Risoluzione delle anomalie e 1FN



- Il problema della relazione EMPLOYEE2 è che contiene dati su due entità: DIPENDENTE e CORSO.
- Useremo la teoria della normalizzazione (descritta più avanti) per dividere EMPLOYEE2 in due relazioni.
- Una delle relazioni risultanti è EMPLOYEE1.
- L'altra la chiameremo EMP COURSE. La chiave primaria di questa relazione è la combinazione di EmpID e CourseTitle
- EMP COURSE è privo dei tipi di anomalie descritte in precedenza e quindi ben strutturato.

EMPLOYEE1			
<u>EmpID</u>	Name	DeptName	Salary
100	Margaret Simpson	Marketing	48,000
140	Allen Beeton	Accounting	52,000
110	Chris Lucero	Info Systems	43,000
190	Lorenzo Davis	Finance	55,000
150	Susan Martin	Marketing	42,000

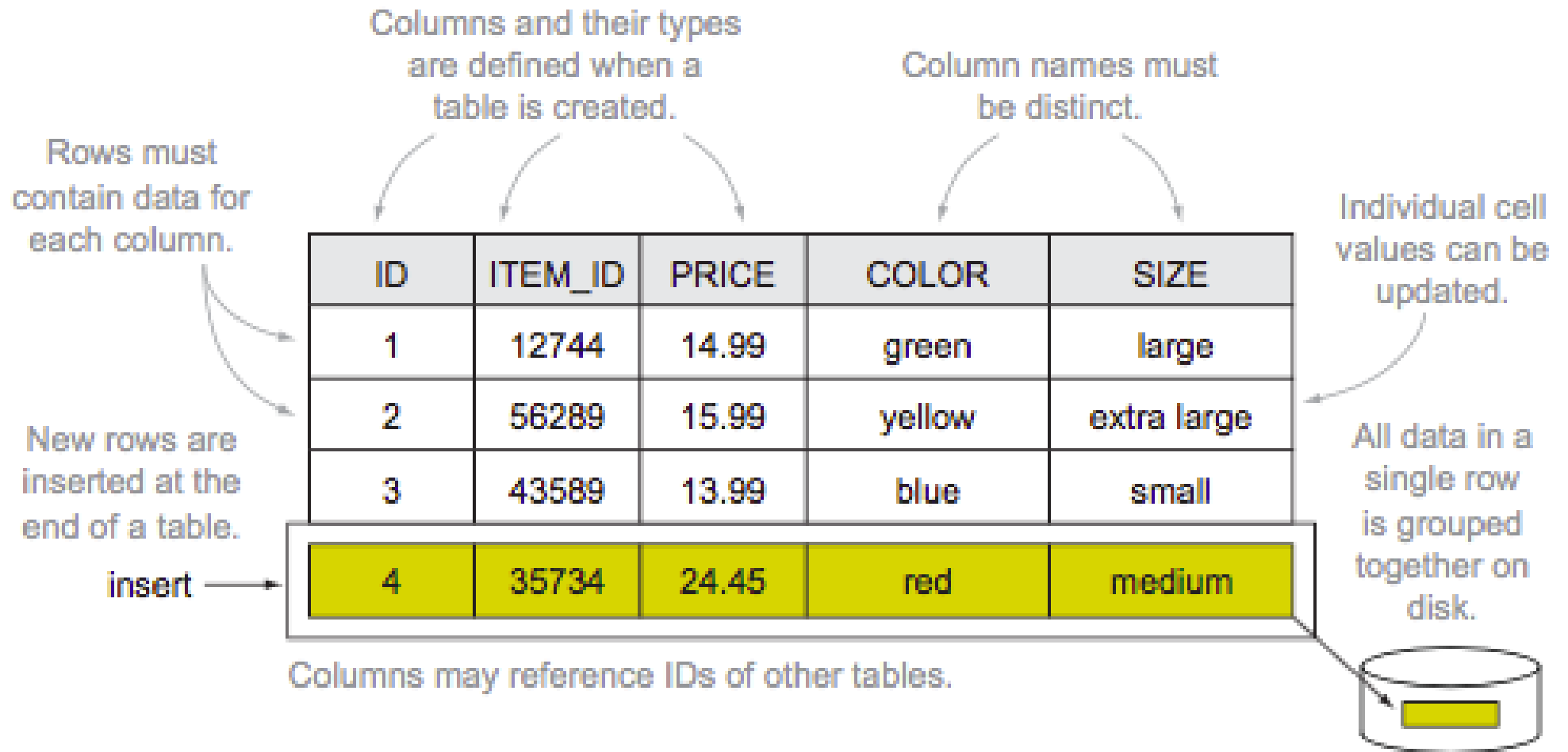
<u>EmpID</u>	<u>CourseTitle</u>	DateCompleted
100	SPSS	6/19/2015
100	Surveys	10/7/2015
140	Tax Acc	12/8/2015
110	Visual Basic	1/12/2015
110	C++	4/22/2015
150	SPSS	6/19/2015
150	Java	8/12/2015



4th Lesson - NoSQL

- From RDBMS to NoSQL

RDBMS – row store



RDBMS - views



We want to restrict
general access to this
column.

Physical table

ORDER_ID	ORDER_DATE	SHIP_STATUS	CARD_INFO	TOTAL
123	2012-07-11	SHIPPED	VISA-1234...	39.45
124	2012-07-12	BACKORDER	MC-5678...	29.37
125	2012-07-13	SHIPPED	AMEX-9012...	42.47

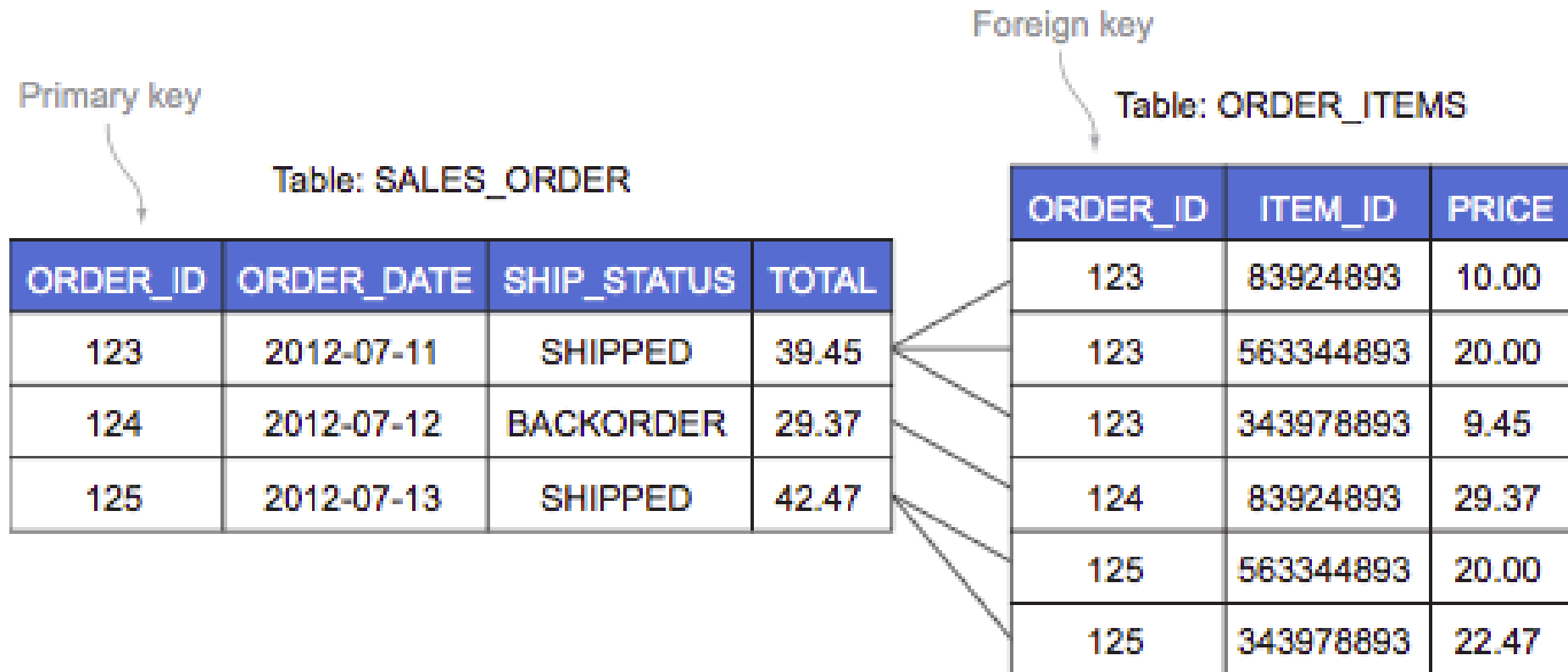
The physical table includes all the column, including credit card info. Only select users ever see the physical table.

View of table

ORDER_ID	ORDER_DATE	SHIP_STATUS	TOTAL
123	2012-07-11	SHIPPED	39.45
124	2012-07-12	BACKORDER	29.37
125	2012-07-13	SHIPPED	42.47

The view excludes some fields like credit card information. All sales analysts have access to the views.

RDBMS - JOINS



- Issues with scaling up when the dataset is just too big
- RDBMS were not designed to be distributed
- Began to look at multi-node database solutions
- Different approaches include:
 - Master-slave
 - Sharding

Scaling RDBMS – Master/Slave vs Sharding



- Master-Slave
 - All writes are written to the master. All reads performed against the replicated slave databases
 - Critical reads may be incorrect as writes may not have been propagated down
 - Large data sets can pose problems as master needs to duplicate data to slaves
- Partition or sharding
 - Scales well for both reads and writes
 - Not transparent, application needs to be partition-aware
 - Can no longer have relationships/joins across partitions
 - Loss of referential integrity across shards

RDBMS Limits



Feature	Strength	Weakness
Joins between tables	New views of data from different tables can easily be created.	All tables must be on the same server to make joins run efficiently. This makes it difficult to scale to more than one processor.
Transactions	Defining begin point, end point, and completion of critical transactions in an application is simple.	Read and write transactions may be slowed during critical times in a transaction unless the transaction isolation level is changed.
Fixed data definitions and typed columns	Easy way to define structure and enforce business rules when tables are created. You can verify on insert that all data conforms to specific rules. Allows range indexes over columns.	Difficult to work with highly variable and exception data when adding to a column.
Fine-grained security	Data access control by row and column can be done with a series of view and grant statements.	Setup and testing security access for many roles can be a complex process.
Document integration	None. Few RDBMSs are designed to easily query document structures.	Difficult to create reports using both structured and unstructured data.

- Not Only SQL or NO! SQL
- **A more general definition... a datastore that does not follow the relational model including using SQL to interact with the data.**
- Why?
 - One size does not fit all
 - Relational Model has scaling issues
 - Freedom from the tyranny of the DBA?

How did we get here?



- Explosion of social media sites (Facebook, Twitter) with large data needs
- Rise of cloud-based solutions such as Amazon S3 (simple storage solution)
- A shift to dynamically-typed data with frequent schema changes
- Open-source community

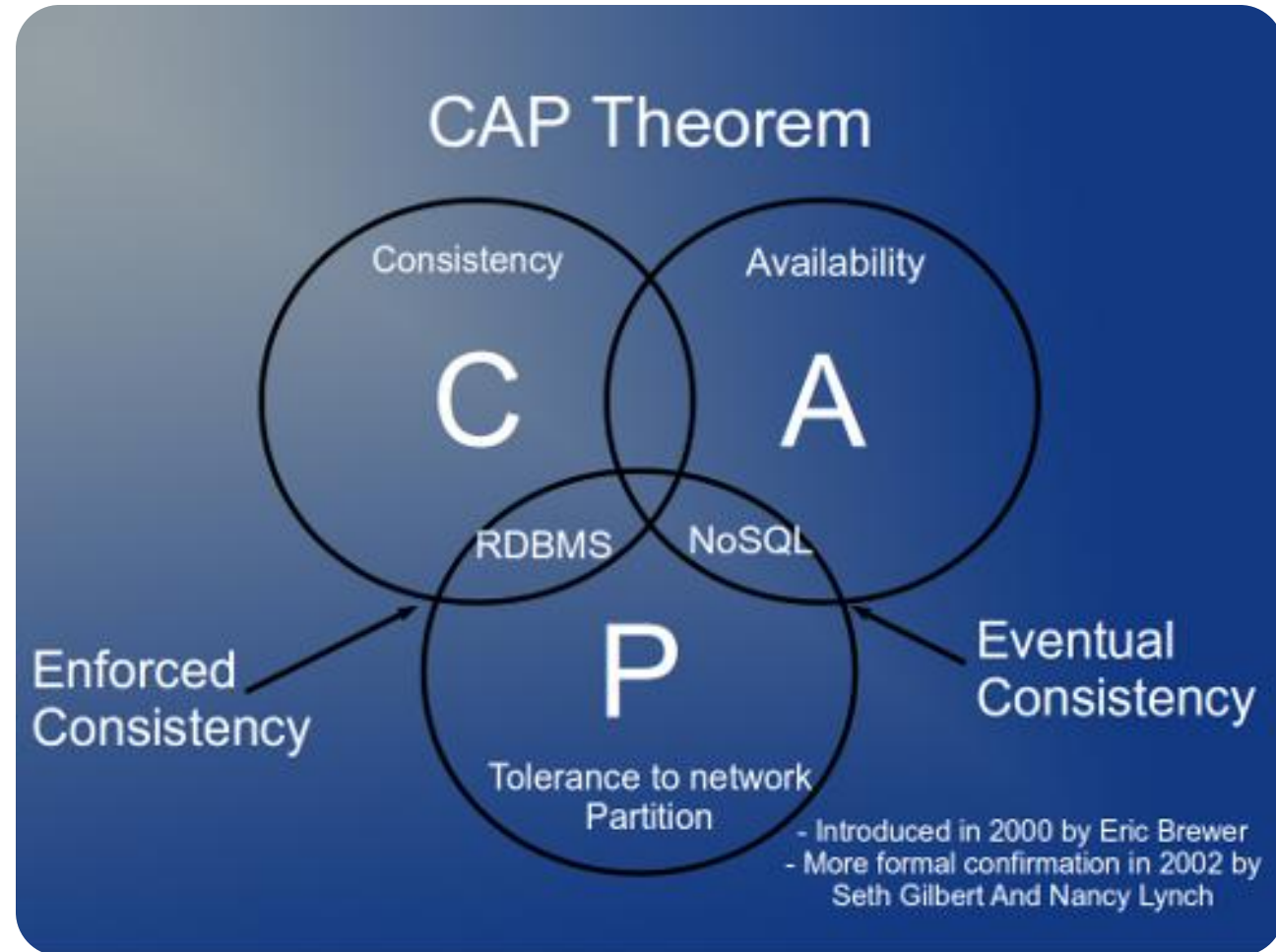
- **Schema-less** data storage
- Non-relational
- Flexible schema
- Other or additional query languages than SQL
- Easily distributed – **horizontal scaling**
- Unstructured data
- Supports **big data**
- Quick data storage and traversal
- Better performance

- Three major papers were the seeds of the NoSQL movement
 - [BigTable \(Google\)](#)
 - [Dynamo \(Amazon\)](#)
 - ❖ Gossip protocol (discovery and error detection)
 - ❖ Distributed key-value data store
 - ❖ Eventual consistency
 - CAP Theorem *Eric Brewer 2000*

CAP theorem



It is impossible for a [distributed data store](#) to simultaneously provide more than two out of the following three guarantees



- **Consistency:** write a value and then you read the value you get the same value back.
- **Availability:** may not always be able to write or read. The system will say you can't write because it wants to keep the system consistent.
- **Partitionability:**
 - Any guarantee of consistency or availability is still guaranteed even if there is a partition.
 - If a system is not partition-tolerant, that means that if the network can lose messages or any nodes can fail, then any guarantee of atomicity or consistency is voided.

To scale you have to partition, so you are left with choosing either high consistency or high availability for a particular system. You must find the right

Consistency Model



- A consistency model determines rules for visibility and apparent order of updates.
- For example:
 - Row X is replicated on nodes M and N
 - Client A writes row X to node N
 - Some period of time t elapses.
 - Client B reads row X from node M
 - Does client B see the write from client A?
 - **For NoSQL, the answer would be: maybe (EVENTUAL CONSISTENCY)**
 - CAP Theorem states: Strict Consistency (**ACID**) can't be achieved at the same time as availability and partition-tolerance.

- **Atomicity**

- Requires that each transaction be "all or nothing": if one part of the transaction fails, then the entire transaction fails;

- **Consistency**

- Ensures that any transaction will bring the database from one valid state to another.

- **Isolation (Concurrency Control)**

- Ensures that the concurrent execution of transactions results in a system state that would be obtained if transactions were executed sequentially, i.e., one after the other.

- **Durability**

- Ensures that once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.

Eventual Consistency



- When no updates occur for a long period of time, eventually all updates will propagate through the system and all the nodes will be consistent
- For a given accepted update and a given node, eventually either the update reaches the node or the node is removed from service
- Known as BASE (**B**asically **A**vailable, **S**oft state, **E**ventual consistency), as opposed to ACID

- BASE stands for these concepts:
- **Basic Availability** allows systems to be temporarily inconsistent so that transactions are manageable. In BASE systems, the information and service capability are “basically available.”
- **Soft-state** recognizes that some inaccuracy is **temporarily** allowed and data may change while being used to reduce the amount of consumed resources.
- **Eventual consistency** means eventually, when all service logic is executed, the system is left in a consistent state.

- The number-one objective is to **allow new data to be stored**, even at the risk of being out of sync for a short period of time. They relax the rules and allow reports to run even if not all portions of the database are synchronized.
- They're *optimistic* in that they assume that eventually all systems will catch up and become consistent.
- BASE systems tend to be **simpler** and **faster** because they don't have to write code that deals with locking and unlocking resources.
- Their mission is to **keep the process moving** and deal with broken parts at a later time.
- BASE systems are ideal for **web storefronts**, where filling a shopping cart and placing an order is the main priority.

Pros and Cons of RDBMS



- RDBMS pros:
 - ACID transactions
 - Fine-grained security on columns and rows using views
 - Most SQL code is portable to other SQL databases, including open source options.
 - Typed columns and constraints will validate data before it's added to the database and increase data quality.
 - Existing staff members are already familiar with entity-relational design and SQL.
- RDBMS cons:
 - The object-relational mapping layer can be complex.
 - Entity-relationship modeling must be completed before testing begins, which slows development.
 - RDBMSs don't scale out when joins are required.
 - **Sharding over many servers can be done but requires application code and will be operationally inefficient.**
 - **Full-text search requires third-party tools.**

Pros and Cons of NoSQL



- NoSQL pros:
 - Loading test data can be done with drag-and-drop tools **before ER modeling is complete.**
 - Modular architecture allows components to be exchanged.
 - **Linear scaling** takes place as new processing nodes are added to the cluster.
 - Lower operational costs are obtained by autosharding.
 - Integrated search functions provide high-quality ranked search results.
 - There's no need for an object-relational mapping layer.
 - It's easy to store high-variability data.
- NoSQL cons:
 - **ACID transactions can be done only within a document at the database level. Other transactions must be done at the application level.**
 - **Document stores don't provide fine-grained security at the element level.**
 - NoSQL systems are new to many staff members and additional training may be required.
 - The **document store has its own proprietary nonstandard query language**, which prohibits portability.
 - The document store won't work with existing reporting and OLAP tools.



3rd Lesson - NoSQL

- NoSQL Storage Kinds

NoSQL Database Types



- **Key-value stores** are the simplest NoSQL databases. Every single item in the database is stored as an attribute name (or 'key'), together with its value. Examples of key-value stores are Riak and Berkeley DB.
- **Wide-column stores** such as Cassandra and HBase are optimized for queries over large datasets, and store columns of data together, instead of rows.
- **Document databases** pair each key with a complex data structure known as a document.
- **Graph stores** are used to store information about networks of data, such as social connections. Graph stores include Neo4J and triple stores like Fuseki.

What kinds of NoSQL



Type	Typical usage	Examples
<i>Key-value store</i> —A simple data storage system that uses a key to access a value	• Image stores • Key-based filesystems • Object cache • Systems designed to scale	• Berkeley DB • Memcache • Redis • Riak • DynamoDB
<i>Column family store</i> —A sparse matrix system that uses a row and a column as keys	• Web crawler results • Big data problems that can relax consistency rules	• Apache HBase • Apache Cassandra • Hypertable • Apache Accumulo
<i>Graph store</i> —For relationship-intensive problems	• Social networks • Fraud detection • Relationship-heavy data	• Neo4j • AllegroGraph • Bigdata (RDF data store) • InfiniteGraph (Objectivity)
<i>Document store</i> —Storing hierarchical data structures directly in the database	• High-variability data • Document search • Integration hubs • Web content management • Publishing	• MongoDB (10Gen) • CouchDB • Couchbase • MarkLogic • eXist-db • Berkeley DB XML

What kinds of NoSQL



- NoSQL solutions fall into two major areas:
 - Key/Value or ‘the big hash table’.
 - ❖ Amazon S3 (Dynamo)
 - ❖ Voldemort
 - ❖ Scalaris
 - Schema-less which comes in multiple flavors, column-based, document-based or graph-based.
 - ❖ Cassandra (column-based)
 - ❖ CouchDB (document-based)
 - ❖ Neo4J (graph-based)
 - ❖ HBase (column-based)

Key-Value stores



- Simple data model that **maps keys** to a **list of values**
- Easy to achieve
 - Performance
 - Fault tolerance
 - Heterogeneity
 - Availability

due to its schema-less data model and fine granularity partitioning of the data

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

Key / Value Store

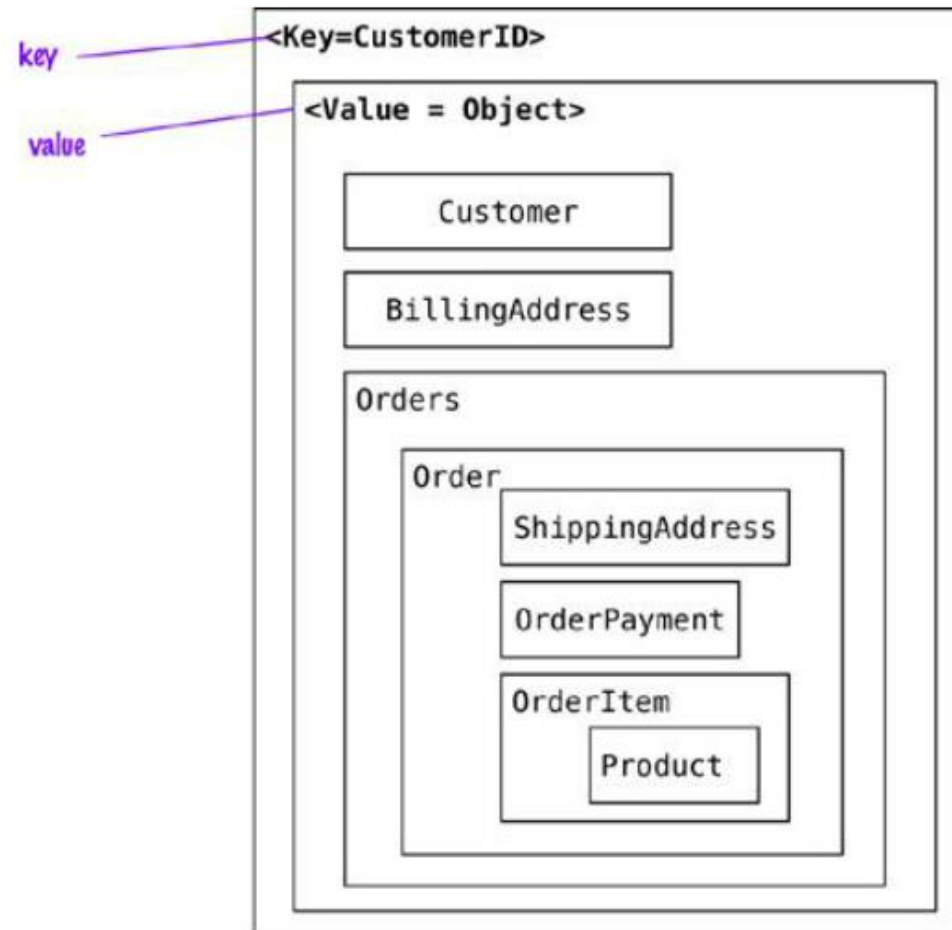


Key / Value store databases allow for values to be associated with and looked up by a key.

Keys can be associated with more than one value.

Data can be stored in the native data type of a particular programming language.

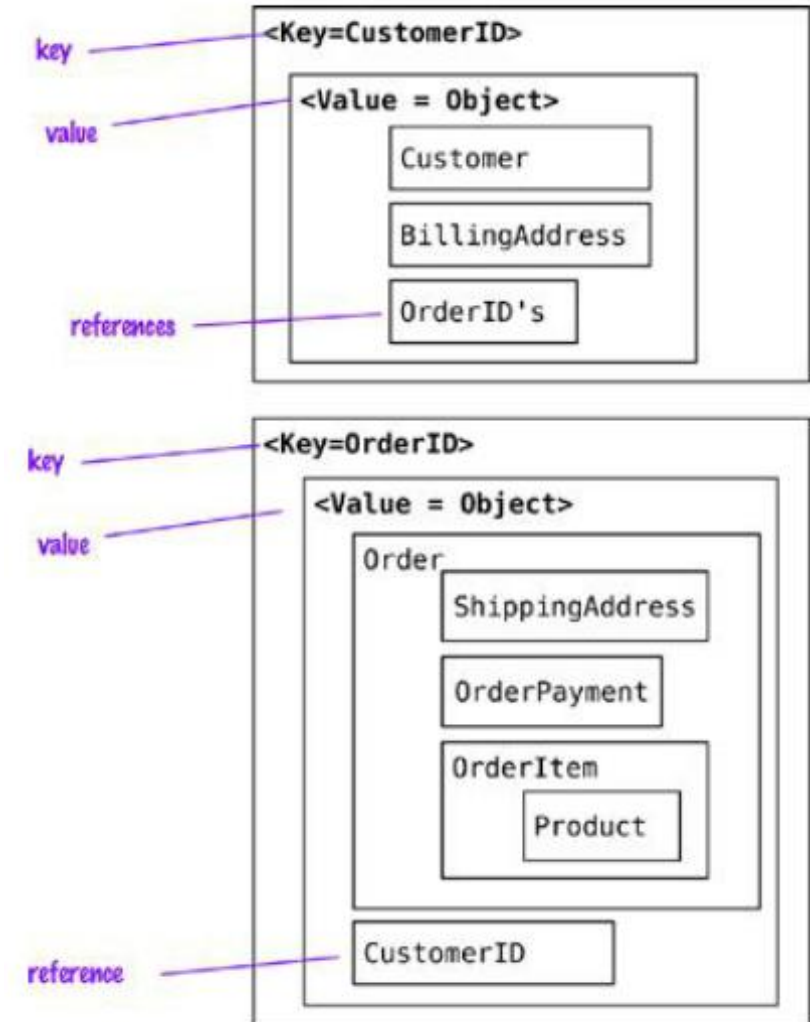
Data Accesses in key-value store



Data Accesses in key-value store



- Splitting Aggregates:
 - we can now find the orders independently from the Customer, and with the orderID reference in the Customer we can find all Orders for the Customer
- Analytics
 - An aggregate update may store which Orders have a given Product in them
 - Useful for Real Time Analytics



Pros:

- very fast
- very scalable
- simple model
- able to distribute horizontally

Cons:

- many data structures (objects) can't be easily modeled as key value pairs

Pros:

- Schema-less data model is richer than key/value pairs
 - ❖ eventual consistency
 - ❖ many are distributed
 - ❖ still provide excellent performance and scalability
- Non-uniform data
- Easily change your data organization

Cons:

- A program that accesses data:
 - ❖ almost always relies on some form of implicit schema
 - ❖ it assumes that certain fields are present
 - ❖ carry data with a certain meaning
- Typically no ACID transactions or joins
- The schema cannot be used to:
 - ❖ decide how to store and retrieve data efficiently
 - ❖ **ensure data consistency**
- The implicit schema is shifted into the application code that accesses data



3° Lesson - NoSQL

- *Column-based Storage*

Column-Family Stores



keyspace

column family

column

name

value

timestamp

Column-Family Stores - Data Model



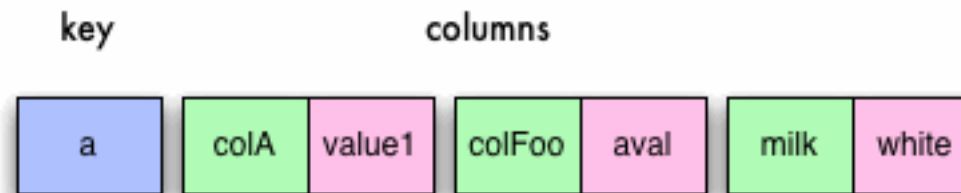
A single column

Name	colA	value1	Value
------	------	--------	-------

Column-Family Stores - Data Model



A single row



Column-Family Stores - Data Model



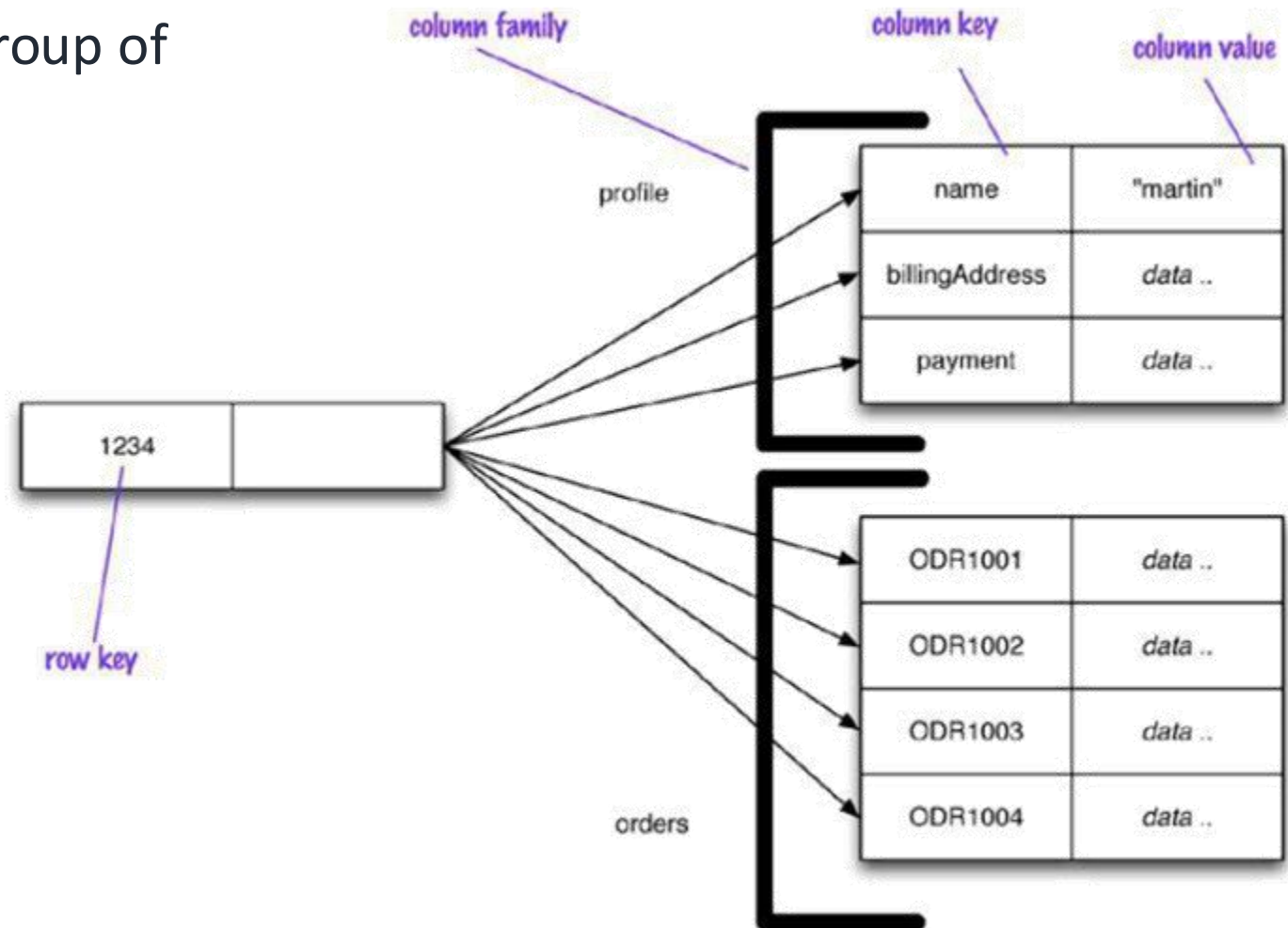
Color Key

- Keys
- Column Names
- Column Values

Column-Family Stores - Data Model



- A two-level aggregate structure:
 - A key and a row aggregate
 - A row aggregate is a group of columns



Column-Family Stores



- Operations also allow picking out a particular column `get('1234', 'name')`
- Each column:
 - has to be part of a single column family
 - acts as unit for access
- You can add any column to any row, and rows can have very different columns
- You can model a list of items by making each item a separate column
- Two ways to look at data:
 - Row-oriented
 - ❖ Each row is an aggregate
 - ❖ Column families represent useful chunks of data within that aggregate
 - Column-oriented:
 - ❖ Each column family defines a record type
 - ❖ Row as the join of records in all column families

Column-Family Stores





3rd Lesson - NoSQL

- *Document-based Storage*

Document databases store information in documents such as JSON or XML.

Document format implies the relationship between data points in the document.

Most documents create hierarchies of data inside themselves.

Key-Value vs Document Store



- Key-value database
 - A key plus a big BLOB of mostly meaningless bits
 - We can store whatever we like in the aggregate
 - We can only access an aggregate by lookup based on its key
- Document database
 - More flexibility in access
 - ❖ we can submit queries to the database based on the fields in the aggregate
 - ❖ we can retrieve part of the aggregate rather than the whole thing
 - Indices based on the contents of the aggregate

Document Stores

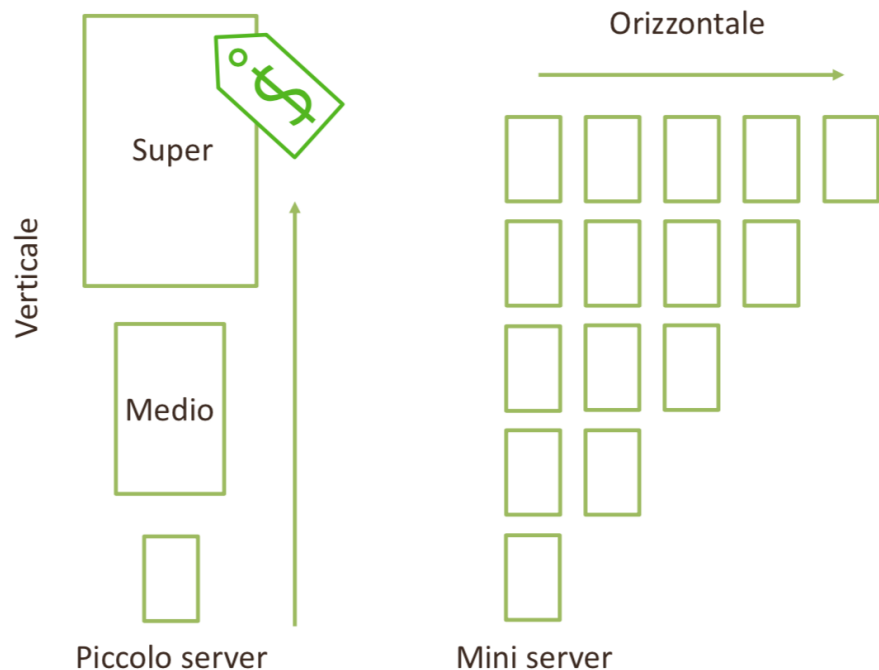




4th Lesson - NoSQL

- *MongoDB*

- Scalabilità
 - ❖ Evoluzione dell'hardware
 - ❖ Big Data
- Facilitare lo sviluppo
 - ❖ Nuove metodologie di sviluppo
- Rappresentazione dei dati intuitiva
 - ❖ Dati semi-strutturati, non strutturati, strutture dati complesse
 - ❖ Polimorfismo



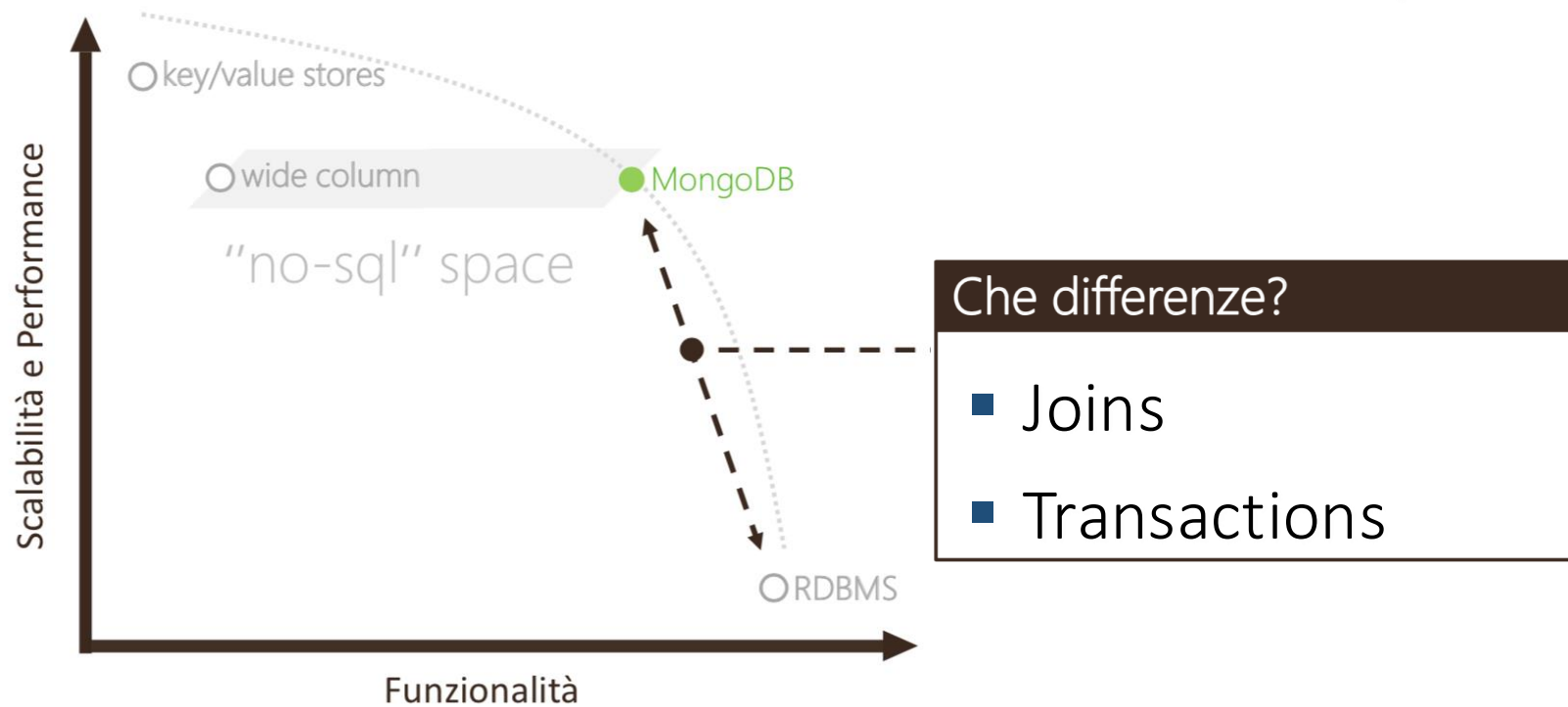
Vantaggi:

- **Fail-hover**, se una macchina smette di funzionare le sue attività vengono indirizzate verso un'altra macchina attiva
- **Alta disponibilità**, assicura la disponibilità del servizio durante varie tipologie di failure
- **Disaster recovery**, garantire un'erogazione continua dei servizi, RPO (recovery point objective)

MongoDB nel panorama dei database



- Key/value stores, come Memcached
- RDBMS, come Oracle, SQL Server, MySQL
- Wide column, come Cassandra



Database NO-SQL

Joins e Transactions in un ambiente distribuito

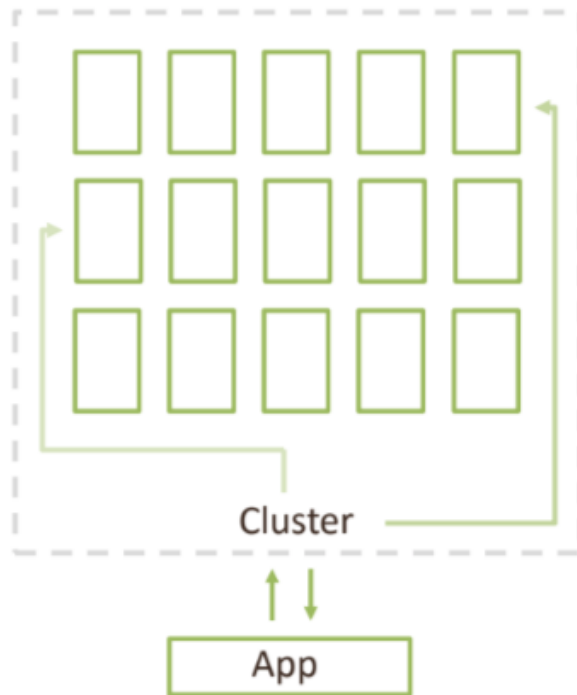


Joins

- Con quale criterio si distribuiscono le tabelle nei vari servers?
- Più server vengono coinvolti più aumentano i tempi di comunicazione

Complex Transactions

- Utili nei DB tradizionali
- Commit / Rollback
- Proprietà ACID (Atomicity, Consistency, Isolation, Durability)



Un nuovo approccio: niente join



Approccio Relazionale

- Le joins sono essenziali
- Impossibile fare di meglio senza

Un nuovo approccio: niente join



Approccio Relazionale

- Le joins sono essenziali
- Impossibile fare di meglio senza

Come fare?

Data Model differente

- Potente
- Agile
- Flessibile

Un nuovo approccio: niente join



Approccio Relazionale

- Le joins sono essenziali
- Impossibile fare di meglio senza

Come fare?

Data Model differente

- Potente
- Agile
- Flessibile

Possibile soluzione

Key/value store?

- Set(k,v) Get(k)
- Troppo semplice, non offre molte funzionalità

Un nuovo approccio: niente join



Approccio Relazionale

- Le joins sono essenziali
- Impossibile fare di meglio senza

Come fare?

Data Model differente

- Potente
- Agile
- Flessibile

Possibile soluzione

Document-Oriented

- Intuitiva rappresentazione dei dati (OOP)
- Adatto alle nuove metodologie di sviluppo

Qualcosa in più

Key/value store?

- Set(k,v) Get(k)
- Troppo semplice, non offre molte funzionalità

Un nuovo approccio: niente join



Approccio Relazionale

- Le joins sono essenziali
- Impossibile fare di meglio senza

Come fare?

Data Model differente

- Potente
- Agile
- Flessibile

Possibile soluzione

Key/value store?

- Set(k,v) Get(k)
- Troppo semplice, non offre molte funzionalità

Qualcosa in più

Document-Oriented

- Intuitiva rappresentazione dei dati (OOP)
- Adatto alle nuove metodologie di sviluppo

Che codifica?

JSON

- Familiare
- Indipendente, standard RFC

Un nuovo approccio: niente join



Approccio Relazionale

- Le joins sono essenziali
- Impossibile fare di meglio senza

Come fare?

Data Model differente

- Potente
- Agile
- Flessibile

Possibile soluzione

Document-Oriented

- Intuitiva rappresentazione dei dati (OOP)
- Adatto alle nuove metodologie di sviluppo

Qualcosa in più

Key/value store?

- Set(k,v) Get(k)
- Troppo semplice, non offre molte funzionalità

Che codifica?

JSON

- Familiare
- Indipendente, standard RFC

Alternativa
alle Joins

Pre-Join / Embedding

- Incapsulazione
- «Denormalizzazione»

Cos'è MongoDB?



name	color
delfino	grigio
pantera	nero

Tabella

tradizionale

Animals

```
{name: "delfino",  
color: "grigio" }
```

Fin qui

nulla di nuovo

```
{name: "delfino",  
color: "grigio",  
oceans: [ "atlantico", "pacifico", "indiano" ] }
```

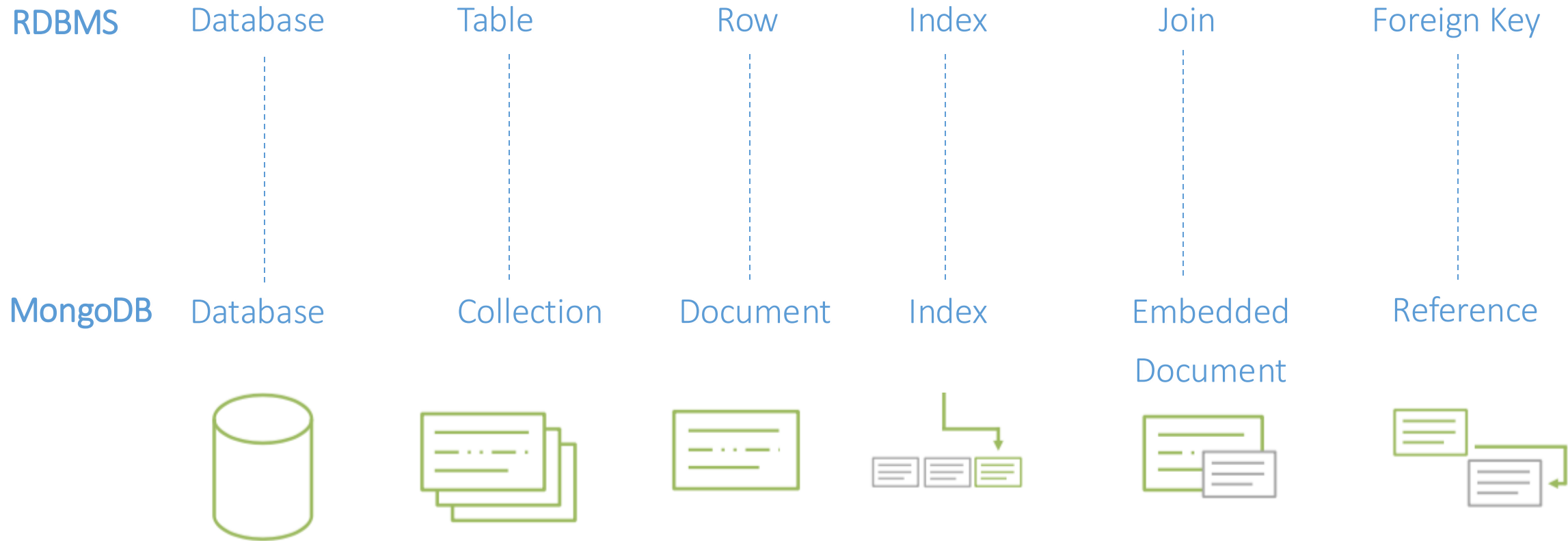
Utile e facilita la programmazione

- Orientato ai documenti (codifica JSON/BSON)
- «Schemaless» / Schema dinamico
- Scalabile
 - Replication / Auto-sharding
- Ricco di funzionalità
 - Text-search aggregation,
 - Geospatial Map reduce

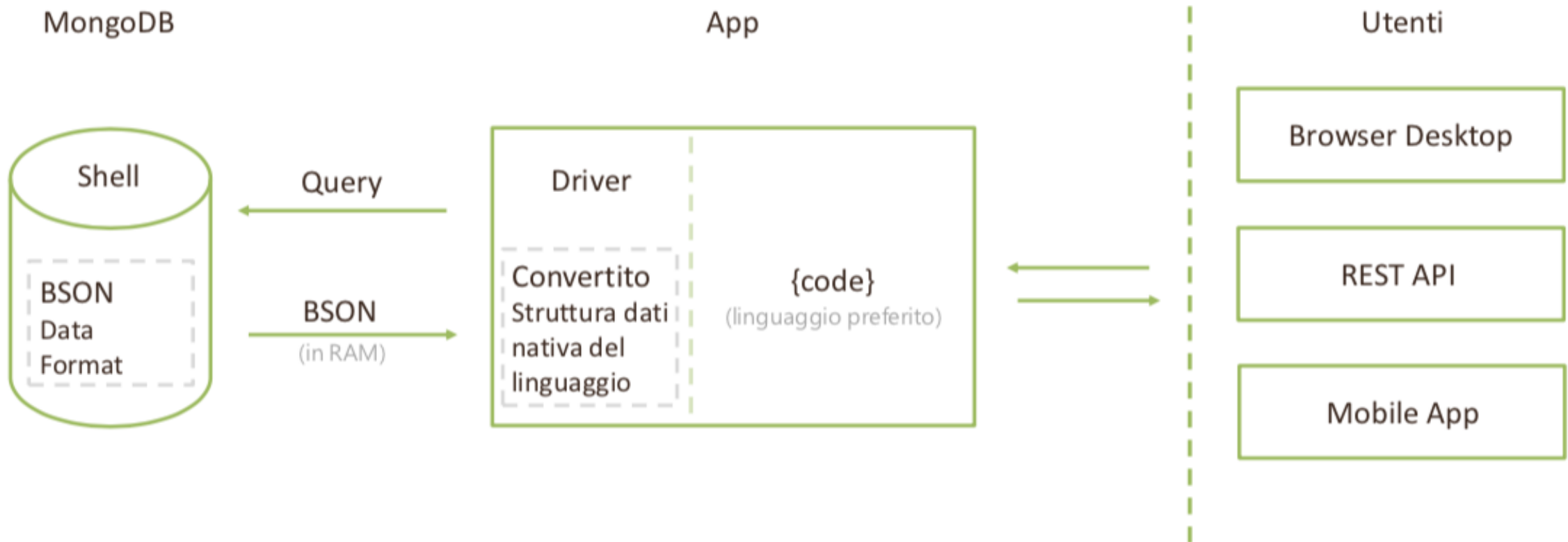
```
{name: 'panter',  
color: 'nero',  
nations: [ "cina", "nepal", "india" ],  
legs:4}
```

Oggetti simili con diversi attributi
(polimorfismo)

Una nuova terminologia



Panoramica: MongoDB in un app



MongoDB = ~~senza schema~~ schema dinamico



Schema interno

- È un catalogo di:
 - Databases
 - ❖ Collection
 - Documents
 - Indexes
- _id e index obbligatori e impliciti

Un vero schema dinamico

- Differenti documenti possono avere differenti schemi (supporta il polimorfismo)
- Risolve la mancata corrispondenza dei dati tra Programmazione ad Oggetti e database
- Le aziende possono adattarsi ai cambiamenti più velocemente (agile, scrum, extreme programming)

Vantaggi di uno schema dinamico

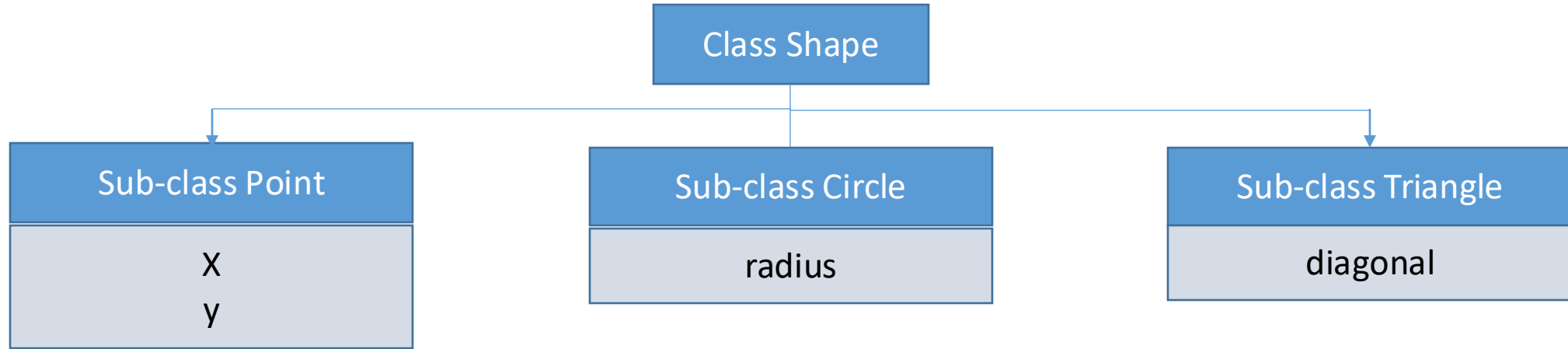


Tabella Shapes

id	shape	x	y	radius
1	Point	4	2	<i>null</i>
2	Circle	<i>null</i>	<i>null</i>	1

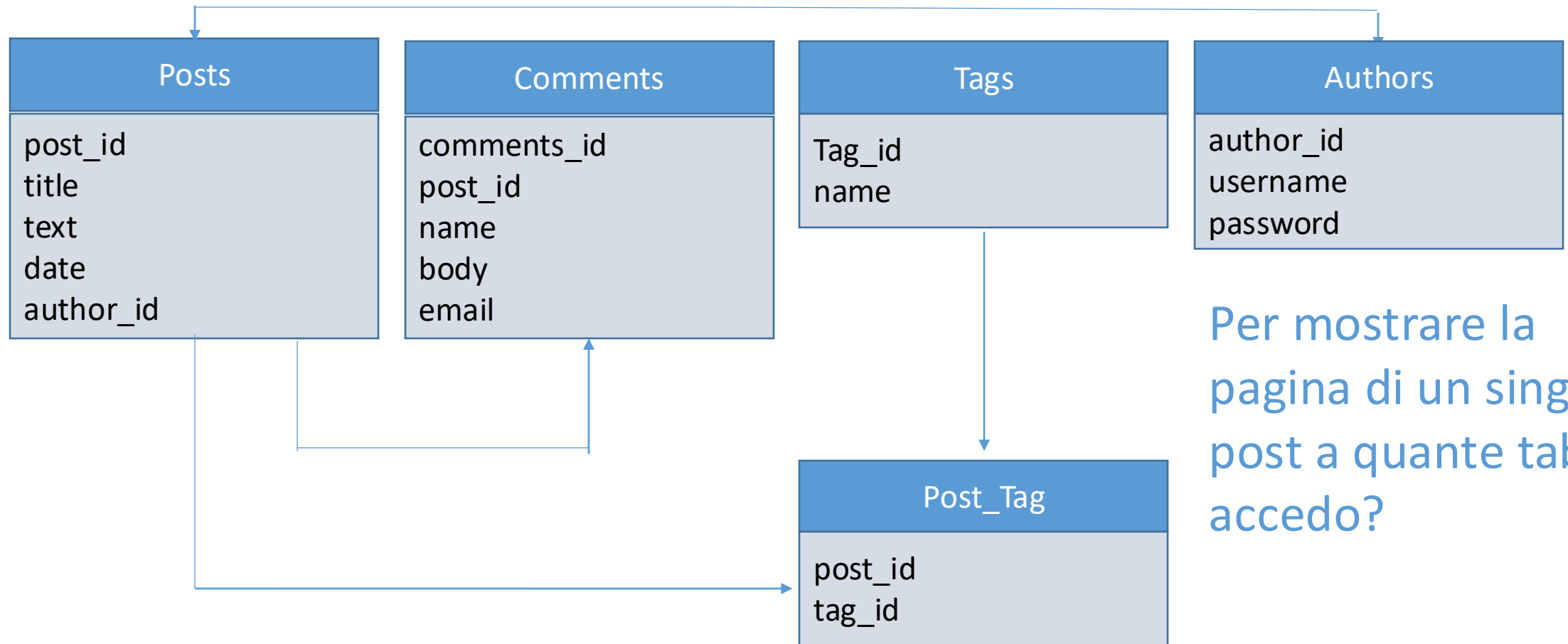
ALTER TABLE

Collection Shapes

```
{shape: «Point»,  
x:4,  
y:3  
}
```

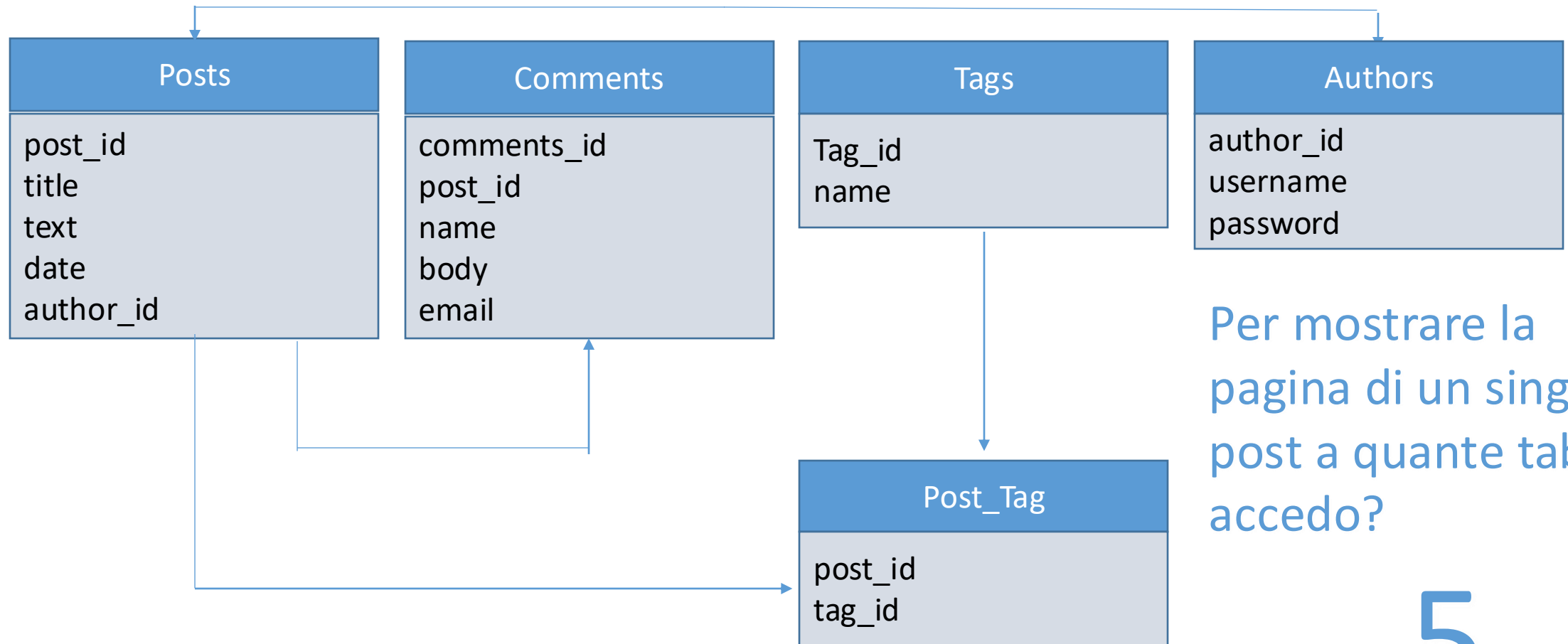
```
{shape: «Circle»,  
radius:1,  
}
```

Un blog in database relazionale, 5 tabelle



Per mostrare la pagina di un singolo post a quante tabelle accedo?

Un blog in database relazionale, 5 tabelle



Per mostrare la pagina di un singolo post a quante tabelle accedo?

5

Un blog in documents, 2 collection



Posts collection

```
{_id: ObjectId(...),
  title: "L'articolo più bello del mondo!",
  body: "bla bla",
  date: ISODate( "2015-08-19T06:01:17.171Z"),
  author: 'GiuseppeFenza',
  comments: [
    { name: 'Giuseppe',
      body: "Lo penso anch'io!",
      email: 'gfenza@unisa.it' },
    { name: "Anonimo",
      body: "Un commento anonimo!", email:
        "email.anonima@gmail.com" } ],
  tags: [ "mongodb", "no-sql", "performance" ] }
```

Authors collection

```
{_id: 'GiuseppeFenza', password: "aterges" }
```

Per mostrare la pagina di un
singolo post a quante
collection accedo?

Pre-Join / Embedding: 2 tabelle in 1 documento



Tabella Clients

ID	name	surname	city
1	Giuseppe	Fenza	Cagliari
2	Mario	Rossi	Cagliari
3	MarBerto	...	...

Tabella Cars

ID	model	year	id_cclient
1	Smart	2010	1
2	Ferrari	1995	1
3	BMW	2015	2



Documento semplice e leggibile

```
{ name: 'Giuseppe',  
  surname: 'Fenza',  
  city: 'Salerno',  
  cars: [  
    { model: "Smart", year: 2010 },  
    { model: "Ferrari", year: 1995 }  
  ],  
  location: {  
    type: "Point",  
    coordinates: [39.216,9.116]  
  },  
}
```


Un blog in documents, 2 collection



Posts collection

```
{_id: ObjectId(...),
  title: "L'articolo più bello del mondo!",
  body: "bla bla",
  date: ISODate( "2015-08-19T06:01:17.171Z"),
  author: 'GiuseppeFenza',
  comments: [
    { name: 'Giuseppe',
      body: "Lo penso anch'io!",
      email: "gfenza@unisa.it" },
    { name: "Anonimo",
      body: "Un commento anonimo!", email:
        "email.anonima@gmail.com" } ],
  tags: [ "mongodb", "no-sql", "performance" ] }
```

Authors collection

```
{_id: 'GiuseppeFenza', password: "aterges" }
```

Per mostrare la pagina di un
singolo post a quante
collection accedo?

1

Embedding o non embedding, questo è il problema



Posts collection

```
{...
  comments: [
    { ... },
    { ... },
  ],
  tags: ["...", "...", "..."]
}
```

Tags

- Potrebbero risultare duplicati in differenti posts
- La sostituzione di un tag diventa tediosa



- Pazienza, mi accontento
- E' una modifica inusuale, poco frequente

Come decidere?

- Seguire il «flusso d'accesso ai dati»
- Quando l'accesso al singolo post si necessita anche dei commenti e tags, ha senso incapsulare
- Limite di 16mb per ogni documento. Nei rari casi in cui si supera tale limite conviene utilizzare una collection separata

Modellazione dei Commenti: incapsulamento



Posts collection

```
{_id: ObjectId(...),  
  title: "L'articolo più bello del mondo!",  
  body: "bla bla",  
  date: ISODate( "2015-08-19T06:01:17.171Z"),  
  author: 'GiuseppeFenza',  
  comments: [  
    { name: 'Giuseppe',  
      body: "Lo penso anch'io!",  
      email: "gfenza@unisa.it" },  
    { name: "Anonimo",  
      body: "Un commento anonimo!", email:  
        "email.anonima@gmail.com" } ],  
  tags: [ "mongodb", "no-sql", "performance" ] }
```

- Una sola query per ottenere tutti i dati
- Si rischia di superare il limite dei 16MB

Modellazione dei Commenti: reference



Posts collection

```
{_id: ObjectId(...),  
  title: "L'articolo più bello del mondo!",  
  body: "bla bla",  
  date: ISODate( "2015-08-19T06:01:17.171Z"),  
  author: "GiuseppeFenza",  
  comments: [  
    ObjectId(...),  
    ObjectId(...),  
    ObjectId(...),  
    ....],  
  tags: [ "mongodb", "no-sql", "performance" ] }
```

```
{_id: ObjectId(...),  
  post_id: ObjectId(...),  
  name: 'Giuseppe',  
  body: "Lo penso anch'io!",  
  email: 'gfenza@unisa.it' }  
}
```

- Quasi estinto il rischio di superare il limite dei 16MB
- Necessari N+1 query per mostrare l'articolo e i commenti, dove N è il numero dei commenti

Modellazione dei Commenti: approccio ibrido



Posts collection

```
{_id: ObjectId(...),  
  title: "L'articolo più bello del mondo!",  
  body: "bla bla",  
  date: ISODate( "2015-08-19T06:01:17.171Z"),  
  author: 'GiuseppeFenza',  
  comments: [  
    { name: 'Giuseppe',  
      body: "Lo penso anch'io!",  
      email: 'gfenza@unisa.it' },  
    ....],  
  comments_count:85,  
  comments_page:2,  
  tags: [ "mongodb", "no-sql", "performance" ] }
```

Array a grandezza fissa

- I 20 commenti più recenti
- Ordinato in base alla data di inserimento

Numero totale commenti

Numero di pagine

Modellazione dei Commenti: approccio ibrido



Comments collection

```
{_id: ObjectId(...),  
  post_id: ObjectId(...),  
  page: 2,  
  count: 25,  
  comments: [  
    { name: 'Giuseppe',  
      body: "Lo penso anch'io!",  
      email: "gfenza@gmail.com" },  
    ...  
  ]  
}
```

Numero della pagina

Numero di commenti presenti nella pagina

Array con massimo 40 commenti

JSON: dati object-style facilmente leggibili



XML

```
<?xml version="1.0" encoding="UTF-8"?>
<response uri="http://fake-url.com">
  <result>
    <Accounts>
      <row no="1">
        <FL val="id">12421600000000072037</FL>
        <FL val="phone"><![CDATA[null]]></FL>
        <FL
val="website"><![CDATA[www.joshuawyse.com]]></FL>
        <FL val="employees"><![CDATA[0]]></FL>
        <FL val="billingStreet"><![CDATA[null]]></FL>
        <FL val="shippingStreet"><![CDATA[null]]></FL>
        <FL val="billingCity"><![CDATA[null]]></FL>
        <FL val="shippingCity"><![CDATA[null]]></FL>
        <FL val="billingState"><![CDATA[null]]></FL>
        <FL val="shippingState"><![CDATA[null]]></FL>
        <FL val="billingCode"><![CDATA[null]]></FL>
        <FL val="shippingCode"><![CDATA[null]]></FL>
        <FL val="billingCountry"><![CDATA[null]]></FL>
        <FL val="shippingCountry"><![CDATA[null]]></FL>
        <FL val="description"><![CDATA[null]]></FL>
      </row>
    </Accounts>
  </result>
</response>
```

JSON

```
{
  "id": "12421600000000072038",
  "description": "3",
  "website": "3",
  "numberOfEmployees": "3",
  "phone": "3",
  "name": "account3",
  "shippingAddress": {
    "country": "3",
    "stateOrProvince": "3",
    "city": "3",
    "postalCode": "3",
    "street1": "3"
  },
  "billingAddress": {
    "country": "3",
    "stateOrProvince": "3",
    "city": "3",
    "postalCode": "3",
    "street1": "3"
  }
}
```

JSON: tipi e sintassi



1. String
2. Number
3. Boolean
4. Null
5. Array
6. Object/Doc

```
{string: "I'm a string!",  
number: 33,  
boolean: true,  
null: null,  
array: [ 5, "20" ],  
object: {  
    string: "Me too!",  
    number: 10  
},  
}
```


JSON: alcune cose da ricordare



1. Lo standard consiglia di usare i " doppi apici nelle chiavi, facoltativo se inizia con una lettera
2. In caso di chiave duplicata viene considerato il valore dell'ultima
3. Solo 2 strutture dati basilari da combinare arbitrariamente

Array

lista ordinata di elementi [..., ..., ...]

Dictionary

mappa associativa { chiave: valore }



1 tipo: stringa

6 tipi

{ "chiave" : valore }

{ name : "Alberto",
name : "Marta" } → name : "Marta"

```
{title : "Nuovo corso fighissimo!",  
attachments : [ {  
    name : "file PDF",  
    files : [ "/slide01.pdf",  
              "/slide02.pdf",  
              "/slide03.pdf" ]  
},  
{ ...} ] }
```

BSON (Binary JSON)



1. Open standard per rappresentare il JSON in formato binario
2. Utilizzato in maniera trasparente all'interno del database per questioni di efficienza
3. Conversione automatica interna attraverso i Drivers

Scansione veloce

```
{_id : ...,  
title : "Articolo molto lungo",  
text : { 200kb },  
active : true  
}
```

Nuovi tipi di dato

1. Date: tipo data
2. BinData: una byte array (sorgenti img, UUID)
3. ObjectID: storage compatto

Conclusions



- Time Series Oriented: InfluxDB, KairosDB.
- NewSQL: MySQL versione 8.0, VoltDB, e altri
- Video suggeriti:
 - **NoSQL Distilled to an hour by Martin Fowler**
<https://www.youtube.com/watch?v=ASiU89GI0F0>
 - **MySQL 8.0: un database SQL+NoSQL per i micro servizi by Marco Carlessi [IT]**
<https://www.youtube.com/watch?v=ztuKUEnO9gY>
 - **Cassandra vs MongoDB vs HBase | Difference Between Popular NoSQL Databases**
<https://www.youtube.com/watch?v=QlqylUeqeis>

MongoDB vs Apache Cassandra



- <https://hevodata.com/learn/apache-cassandra-vs-mongodb/>

MongoDB vs Elasticsearch







Hands on MongoDB

Database = BigData2021DB



Create and switch to a new database

```
> use [databasename]
```

Insert a document: insert a document into a collection (e.g., named mycoll). The operation will create the collection if it does not exist yet.

```
> db.mycoll.insert(...)
```

```
db.unicorns.insert({name: 'Aurora', gender:  
    'f', weight: 450})
```


Selectors



Find documents: the `find()` method issues a query to retrieve data from a collection. All queries have the scope of a single collection.

- Queries can return all documents or only those matching a specific filter or criteria
- The `find()` method returns results in a cursor (an iterable object that yields documents)

```
> db.mycoll.find()
```

```
# filter the documents using the query operators {...}
```

```
> db.mycoll.find({ ... })
```

<, >, Not Equal, OR, AND



Exact match

```
> db.mycoll.find({"price" : 300 })
```

Comparison (eq, gt, gte, lt, lte, in, nin):

```
> db.mycoll.find({"price" : { $gt: 300 } })
```

```
> db.mycoll.find({"year" : { $in: [2012, 2016] } })
```

Existence (if document contains a field):

```
> db.mycoll.find({"discount" : { $exists: true } })
```

logical (and, or, not, nor):

AND:

```
> db.mycoll.find({field1 : {...}, field2 : {...} })
```

OR:

```
> db.mycoll.find({  
  $or: [{...}, {...}]  
})
```

<, >, Not Equal, OR, AND



```
db.unicorns.find({gender: 'm',  
                  weight: {$gt: 700}})  
//oppure (non è la scelta migliore,  
//ma vale come esempio)  
db.unicorns.find({gender: {$ne: 'f'},  
                  weight: {$gte: 701}})
```

```
db.unicorns.find({  
                  vampires: {$exists: false}})
```

```
db.unicorns.find({  
                  loves: {$in: ['apple', 'orange']}})
```

<, >, Not Equal, OR, AND



```
db.unicorns.find({gender: 'f',  
  $or: [{loves: 'apple'},  
    {weight: {$lt: 500}}]})
```

Update



Update a document: using `update()`; several update operators are available in mongo.

\$set sets the value of a field in a document. The update can be applied to one or multiple occurrences that matches the update filter.

```
> db.mycoll.update(  
  { field : value },  
  { $set:  
    { "address.street": "East 31st Street" }  
  } )
```

← update filter

Update multiple occurrences

```
> db.mycoll.update(  
  { field : value },  
  { $set: { ... } },  
  {multi: true} )
```

Delete di un campo



```
db.mioraccolta.update(  
  { chiave: "valore" },  
  { $unset: { campoDaCancellare: "" } },  
  { multi: true }  
)
```

Sort query results: to specify an order for the result set, append the **sort()** method to the query.

- Pass to sort() a document which contains the field(s) to sort by and the corresponding sort type (1 for ascending, -1 for descending)

```
> db.mycoll.find().sort( { "name" : 1 } )
```

Remove documents or collection



Remove documents: the `remove()` method removes documents from a collection. The method takes a conditions document that determines the documents to remove

```
> db.mycoll.remove(  
  { "borough": "Manhattan" } )  
  
> db.mycoll.remove(  
  { "borough": "Queens" },  
  { justOne: true } )  
  
# remove all documents:  
> db.mycoll.remove( { } )
```

Drop a collection: to remove all documents from a collection (and the collection itself), the `drop()` operation should be used.

```
> db.mycoll.drop()
```


Aggregation Pipeline



Collection

```
db.orders.aggregate( [  
  $match stage → { $match: { status: "A" } },  
  $group stage → { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }  
)
```

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
{ cust_id: "A123", amount: 300, status: "D" }

orders

\$match

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }

\$group

{ _id: "A123", total: 750 }
{ _id: "B212", total: 200 }

Beyond the lesson on MongoDB



- Big Data Processing via Connector for Hadoop
- Atomic Transaction like \$set, \$inc, findAndModify
- Geospatial Query and Indexes
<https://docs.mongodb.com/manual/geospatial-queries/>
- Full Text Search \$text operator powered by Apache Lucene
<https://docs.mongodb.com/manual/text-search/>
- Indexes

Esercitazione MongoDB - Restaurants



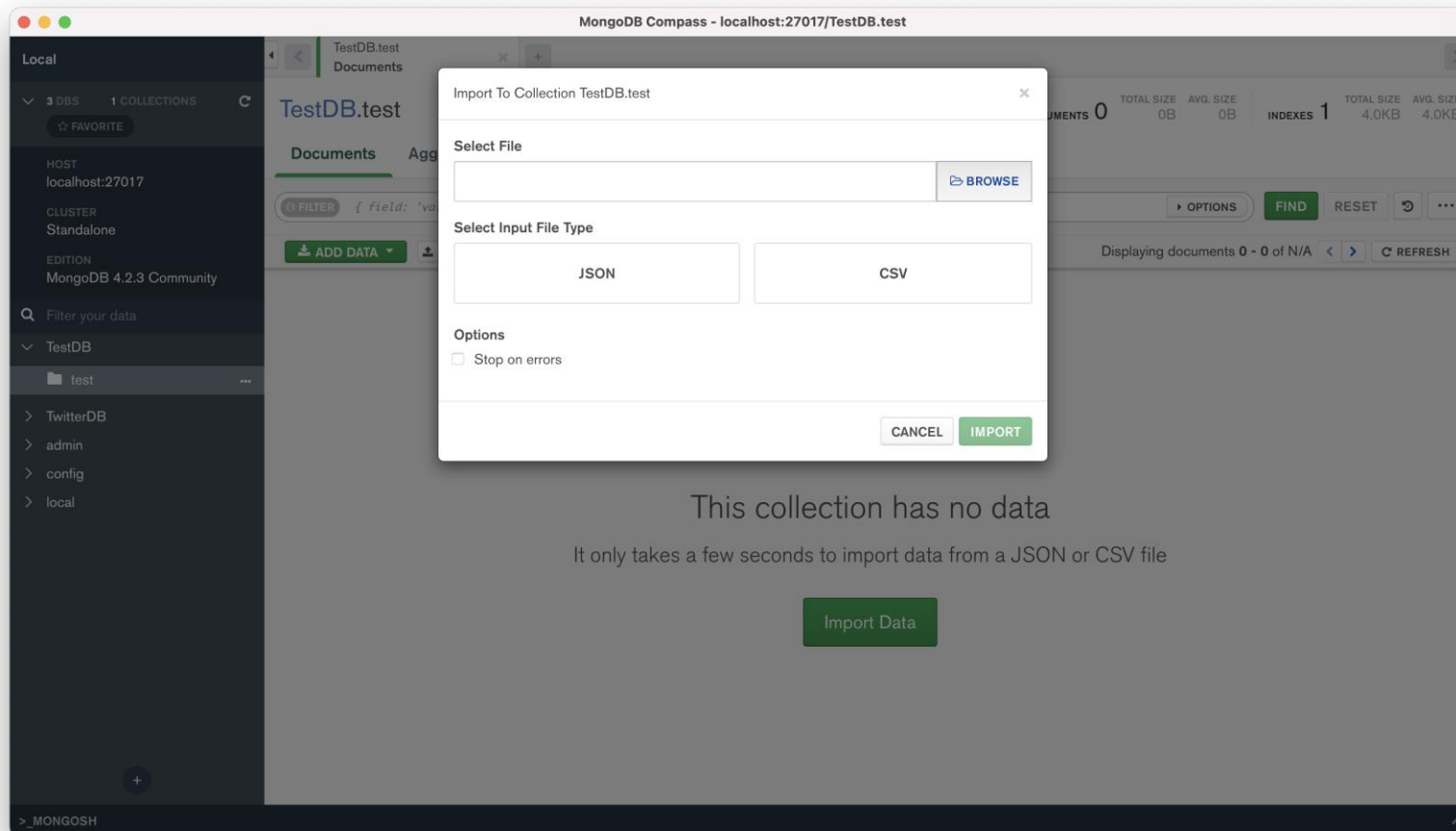
- <https://www.w3resource.com/mongodb-exercises/>
- Si supponga di avere la seguente collection «restaurants»

```
{
  "address": {
    "building": "1007",
    "coord": [ -73.856077, 40.848447 ],
    "street": "Morris Park Ave",
    "zipcode": "10462"
  },
  "borough": "Bronx",
  "cuisine": "Bakery",
  "grades": [
    { "date": { "$date": 1393804800000 }, "grade": "A", "score": 2 },
    { "date": { "$date": 1378857600000 }, "grade": "A", "score": 6 },
    { "date": { "$date": 1358985600000 }, "grade": "A", "score": 10 },
    { "date": { "$date": 1322006400000 }, "grade": "A", "score": 9 },
    { "date": { "$date": 1299715200000 }, "grade": "B", "score": 14 }
  ],
  "name": "Morris Park Bake Shop",
  "restaurant_id": "30075445"
}
```

Import via CLI



- `mongoimport --db restaurants_db --collection restaurants --drop --file ./restaurants.json`



Esercizio 1



- Scrivere una query MongoDB per visualizzare i primi 5 ristoranti che si trovano nel distretto del Bronx.

```
db.restaurants.find({"borough": "Bronx"}).limit(5);
```

Esercizio 2



- Scrivere una query MongoDB per trovare i ristoranti che hanno ottenuto un punteggio superiore a 80 ed inferiore a 100.

```
db.restaurants.find({grades : { $elemMatch:{ "score":{ $gt : 80 , $lt :100}}}});
```

The [\\$elemMatch](#) operator matches documents that contain an array field with at least one element that matches all the specified query criteria.

Esercizio 3



- Scrivere una query MongoDB per trovare i ristoranti che non offrono cucina "americana", che hanno raggiunto un livello "A", e non appartengono al distretto di Brooklyn. I documenti devono essere visualizzati in ordine decrescente rispetto alla cucina.

```
db.restaurants.find( {  
    "cuisine" : {$ne : "American "},  
    "grades.grade" : "A",  
    "borough" : {$ne : "Brooklyn"}  
}  
).sort({"cuisine":-1});
```

Pymongo:

```
db.restaurants.find( {  
    "cuisine" : {"$ne" : "American "},  
    "grades.grade" : "A",  
    "borough" : {"$ne" : "Brooklyn"}  
}  
).sort("cuisine",-1);
```

Esercizio 4



- Scrivere una query MongoDB per trovare l'ID, il nome, il borgo e la cucina dei ristoranti che contengono "Reg" nel nome.

```
db.restaurants.find(  
    { "name": /. *Reg. */ },  
    { "restaurant_id" : 1, "name":1, "borough":1, "cuisine" :1 }  
);
```

Pymongo:

```
db.restaurants.find(  
    { "name": {"$regex": "/*Reg/*"} },  
    { "restaurant_id" : 1, "name":1, "borough":1, "cuisine" :1 }  
);
```


Esercizio 5



- Scrivere una query MongoDB per trovare l'ID, il nome e i voti dei ristoranti che hanno ottenuto un voto "A" e un punteggio 11 in data "2014-08-11T00:00:00Z", tra le date del sondaggio.

Esercizio 6



- Scrivere una query MongoDB per trovare l'ID, il nome, l'indirizzo e la posizione geografica dei ristoranti in cui il 2° elemento dell'array coord contiene un valore che è maggiore di 42 e minore uguale di 52.

Esercizio 7



- Scrivere una query MongoDB per conoscere gli indirizzi che contengono la strada.

Esercizio 8



- Scrivere una query MongoDB che seleziona l'ID, il nome e i voti per quei ristoranti il cui punteggio diviso 7 restituisce 0 come resto.

Esercizio 9



- Scrivere una query MongoDB per trovare l'ID, il nome e i voti dei ristoranti che hanno ottenuto un voto in una data compresa tra "2014-08-10T00:00:00Z" e "2014-08-12T00:00:00Z".

Esercizio 10



- Contare il numero di ristoranti in ogni distretto aventi come cucina quella irlandese

Esercizio 11



- Cerca tutti i ristoranti il cui primo voto è compreso fra 50 e 90 e ordina i risultati in modo decrescente

Esercizio 12



Crea una query aggregata sulla collezione `restaurants` che utilizza **\$unwind** sul campo `grades` per ottenere un documento per ogni valore dell'array `grades` per ogni documento in `restaurants`.

Operatore \$unwind



- L'operatore **\$unwind** prende un elenco di oggetti all'interno di un array e li esplode in modo che ognuno di essi diventi un documento separato. In altre parole, se hai un array all'interno dei tuoi documenti e desideri trattare ciascun elemento di quell'array come un documento separato, puoi usare **\$unwind**.

```
[
  {
    "_id": 1,
    "nome": "Alice",
    "città": ["New York", "Los Angeles",
    "Chicago"]
  },
  {
    "_id": 2,
    "nome": "Bob",
    "città": ["Naples", "Salerno"]
  }
]
```

db.studenti.aggregate([{ \$unwind: "\$città" }])

Provate su <https://mongoplayground.net/>

Esercizio 13



- Cerca tutti i ristoranti il cui lo zipcode è compreso tra 10000 e 10500

Esercizio 14



- Creare la collezione menu, inserisci due documenti ed esegui una query aggregata che utilizza l'operatore **\$lookup** per fare match tra il campo *restaurants._id* e *menu.restaurant_id*. Infine utilizza **\$project** per mostrare solo il menu

Per inserire più documenti utilizza il comando `insertMany`

```
db.menu.insertMany([
{
  "_id" : ObjectId("626d03cd8777cfb20860c630"),
  "restaurant_id" : ObjectId("626c4e305fa1eeeb3da8522e"),
  "menu_items" : [
    {
      "name" : "Spaghetti",
      "price" : 8.5
    },
    {
      "name" : "Rice",
      "price" : 6.5
    }
  ]
},
ADD OTHER MENU DOCUMENTS IF YOU WANT HERE....
])
```

```
db.restaurants.aggregate([
{
  $match: {
    $and:[
      {
        'grades.0.score': {$gte: 50}
      },
      {
        'grades.0.score': {$lt: 90}
      }
    ]
  },
{
  $lookup: {
    from: 'menu',
    localField:'_id',
    foreignField: 'restaurant_id',
    as: 'restaurant_menu'
  },
{
  $project: {'restaurant_menu': 1}
}
])
```

Esempio di utilizzo di *\$out*



```
db.books.insertMany([
  { "_id" : 8751, "title" : "The Banquet", "author" : "Dante", "copies" : 2 },
  { "_id" : 8752, "title" : "Divine Comedy", "author" : "Dante", "copies" : 1 },
  { "_id" : 8645, "title" : "Eclogues", "author" : "Dante", "copies" : 2 },
  { "_id" : 7000, "title" : "The Odyssey", "author" : "Homer", "copies" : 10 },
  { "_id" : 7020, "title" : "Iliad", "author" : "Homer", "copies" : 10 }
])
```

```
db.books.aggregate( [
  { $group : { _id : "$author", books: { $push: "$title" } } },
  { $out : "authors" }
])
```

Esercizio 15



Esegui una query con l'operatore **\$out** sulla collezione restaurant con lo scopo di estrapolare tutti i ristoranti aventi "borough": "Brooklyn" e metterli in una collezione esterna chiamata brooklyn_restaurants

Esercitazione MongoDB - Books



- Si supponga di avere la seguente collection «books»

```
{
  _id:ObjectId("5fb29ae15b99900c3fa24292"),
  title:"MongoDb Guide",
  tag:["mongodb","guide","database"],
  n:100,
  review_score:4.3,
  price:[{v: 19.99, c: "€", country: "IT"},
  {v: 18, c: "£", country:"UK"} ],
  author: {
    _id: 1,
    name:"Mario",
    surname: "Rossi"
  }
}, ...
```

```
{
  _id:ObjectId("5fb29b175b99900c3fa24293",
  title:"Developing with Python",
  tag:["python","guide","programming"],
  n:352,
  review_score:4.6,
  price:[{v: 24.99, c: "€", country: "IT"},
  {v: 19.49, c: "£", country:"UK"} ],
  author:
  {
    _id: 2,
    name:"John",
    surname: "Black"
  }
}, ...
```

Esercizio 1



- Trovare tutti i libri con numero di pagine maggiore di 250

Esercizio 2



- Trovare tutti i libri scritti da «Mario Rossi»

Esercizio 3



- Trovare tutti i libri con prezzo inferiore a 20€ per l'Italia

Esercizio 4



- Trovare prezzo massimo, minimo e medio di tutti i libri con il tag «database»

Esercizio 5



- Contare i libri scritti da «Mario Rossi»

- Per supportare la ricerca testuale in contenuti di tipo stringa, si utilizza **Text**:
 - `db.book.createIndex( { comments: "text" } )`
- I campi di tipo «text» vengono tokenizzati e stemmati quando indicizzati
- La definizione di una lingua di riferimento (**default_language**) può essere utile per una migliore gestione del testo (stemming, stopword removal)
 - Es. `db.book.createIndex( { comments: "text" } , {default_language: "english"})`
- Quando la lingua non è specificata, si applica soltanto la tokenizzazione

Esempio



- Cercare i libri nel cui commento è citata la parola «interessante» o «avvincente»

```
db.book.find({  
  $text:  
    {$search: "interessante avvincente"}  
})
```

Esempio



- Per cercare una intera frase si usano le virgolette:

```
db.book.find({  
  $text:  
    {$search: "\"libro avvincente\""}  
})
```

MongoDB Sample Dataset



- <https://github.com/neelabalan/mongodb-sample-dataset>

Dataset Name	Description
Sample AirBnB Listings Dataset	Contains details on AirBnB listings.
Sample Analytics Dataset	Contains training data for a mock financial services application.
Sample Geospatial Dataset	Contains shipwreck data.
Sample Mflix Dataset	Contains movie data.
Sample Supply Store Dataset	Contains data from a mock office supply store.
Sample Training Dataset	Contains MongoDB training services dataset.
Sample Weather Dataset	Contains detailed weather reports.



Esercitazione

Energy Dataset di Our World in Data



- Il dataset contiene tutte le informazioni sul consumo di energia a livello globale.
- Ogni riga rappresenta i consumi annuali per ogni singolo stato.
- Sono raccolti i dati dal 1900 al 2021.
- Le features in totale sono 122.
- Le prime 10 features sono elencate qui di seguito con la descrizione.

column	description
iso_code	ISO 3166-1 alpha-3 three-letter country codes
country	Geographic location
year	Year of observation
coalprodchange_pct	Annual percentage change in coal production
coalprodchange_twh	Annual change in coal production, measured in terawatt-hours
gasprodchange_pct	Annual percentage change in gas production
gasprodchange_twh	Annual change in gas production, measured in terawatt-hours
oilprodchange_pct	Annual percentage change in oil production
oilprodchange_twh	Annual change in oil production, measured in terawatt-hours
energyconschange_pct	Annual percentage change in primary energy consumption

LINK AL DATASET: <https://github.com/owid/energy-data>

LINK ALLA DESCRIZIONE DELLE FEATURES: <https://www.kaggle.com/datasets/pralabhpoudel/world-energy-consumption>

Mostrare l'incremento del consumo e della produzione di gas rispetto all'anno precedente per l'anno 2021 per Italia, Europa e Mondo.

Aiuto: La percentuale è un valore già presente nel dataset.

Esercitazioni



Risultato:

country	year	gas_cons_change_pct	gas_prod_change_pct
 Europe	 2021	 7.514	 6.547
 Italy	 2021	 7.212	 -18.608
 World	 2021	 4.99	 4.541

Query:

```
db.getCollection("esempiomaster").find(  
  {  
    "year": {$gt: 2020},  
    "country": {  
      "$in": [  
        "World",  
        "Europe",  
        "Italy"  
      ]  
    }  
  }, {gas_cons_change_pct:1, country:1, year:1, gas_prod_change_pct:1}  
).sort({"gas_cons_change_pct": -1.0}).limit(10)
```